

claude-windows / dbc4cdae-04f6-4d06-af90-397bbce3fc57

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\subagents\agent-a60d98a37b4e667dc.jsonl`  
- SHA-256: `129e0102486a87aabdac7b1facc0f1d71f49070bbfad9275030ec7a349ede9d4`  
- Source modified: `2026-07-06T22:01:38+00:00`  
- Imported at: `2026-07-08T15:59:36+00:00`  
- project: `subagents`  
- session_id: `dbc4cdae-04f6-4d06-af90-397bbce3fc57`
```

Transcript

1. user (2026-07-06T21:59:33.538Z)

Read-only investigation in the repo at C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer. I need the EXACT windowing configuration that the "default" (== "milestone") preset uses to turn a WASB trajectory CSV into rally windows ? the one that produces `eval\_baselines/nightly\_baseline.json`'s `configs.default` numbers (e.g. GX010128 f1@0.5=0.582, aggregate f1@0.5=0.472). My local scorer currently uses the raw `(velocity\_thresh=0.02, merge\_gap=2.0, min\_window\_duration=2.0)` which is degenerate (baseline scores ~0.06). I need the production preset so I can score a new trajectory apples-to-apples.

Investigate and answer with file:line + concrete values:

1. `backend/pipeline/detectors/tracknet\_runner.py::trajectory\_to\_action\_windows` ? the FULL parameter list (all kwargs, defaults). Does it support "cap" (segment cap / R2 / R4), "inactivity", "dead_time", "over-seg guard", or similar params beyond velocity_thresh/merge_gap/min_window_duration? List every knob it accepts.
2. How is the "default"/"milestone" preset defined for the nightly baseline? Look in `backend/eval/rally\_seg\_eval.py`, `run\_regression.py`, `run\_eval.py`, `eval\_baselines/nightly\_baseline.json` (its `configs.default` may embed the config/knobs), `config.json` (indexing.wasb / windowing section), and any "preset"/"R2"/"R4"/"cap"/"milestone" definitions. Find the EXACT knob values the `default` preset passes to `trajectory\_to\_action\_windows`.
3. Give me the precise call: `trajectory\_to\_action\_windows(points, fps=59.94, frame\_width=1920, <the default-preset kwargs=values>)` that would reproduce the `default` preset windowing. If some knobs come from `config.json`/WasbConfig, give those exact values.
4. Also confirm: `backend/config/models.py` WasbConfig (or WasbIndexConfig) fields that control windowing and their defaults, and which of those the "default preset" overrides.

Return a concise report: the full `trajectory\_to\_action\_windows` signature, the exact default-preset kwargs+values, and the one-line call to reproduce it. Flag anything you can't pin down.

2. assistant (2026-07-06T21:59:39.090Z)

I'll investigate the windowing configuration for the "default"/"milestone" preset. Let me start by pulling the repo fresh and locating the key files.

3. user (2026-07-06T21:59:44.570Z)

true
Already up to date.

4. user (2026-07-06T21:59:45.491Z)

```
./artifacts/cloudrun_lightup/config.json  
./backend/config/models.py  
./backend/eval/rally_seg_eval.py  
./backend/pipeline/detectors/tracknet_runner.py  
./config.json
```

```
./eval_baselines/nightly_baseline.json
./run_eval.py
./run_regression.py
```

5. assistant (2026-07-06T21:59:50.743Z)

Now let me read the key files. Let me start with the core function and the baseline JSON.

6. user (2026-07-06T21:59:52.367Z)

```
311:     def trajectory_to_action_windows(
```

7. user (2026-07-06T21:59:53.162Z)

```
1      {
2      "_doc": "Frozen nightly rally-quality regression baseline. Regenerate with `python
scripts/nightly_regression.py --update-baseline` after an intended improvement or a corpus
change. Tied to the LOCAL golden corpus (not committed).",
3      "configs": {
4      "baseline_off": {
5      "aggregate": {
6      "dend_abs_median": 41.77921047714383,
7      "dstart_abs_median": 42.467243955066174,
8      "f1@0.25": 0.39981006941991765,
9      "f1@0.5": 0.2366145803119009,
10     "merge_rate": 0.6380248744753217,
11     "precision": 0.2577557018251538,
12     "recall": 0.22719343607869802,
13     "segment_count_ratio": 0.5824947574930804,
14     "split_rate": 0.0,
15     "tol_f1": 0.1382064040388172,
16     "tol_precision": 0.14536686157033415,
17     "tol_recall": 0.13445461315285676
18     },
19     "iou": 0.5,
20     "min_crossings": 0,
21     "n": 15,
22     "note": "Pre-R1-flip all-OFF windowing \u2014 the historical floor; pins the value
the milestone buys.",
23     "per_video": {
24     "Badminton_BXH_2": {
25     "f1@0.25": 0.0,
26     "f1@0.5": 0.0,
27     "merge_count": 3,
28     "merge_rate": 1.0,
29     "num_gt": 44,
30     "num_pred": 4,
31     "split_count": 0,
32     "tol_f1": 0.0
33     },
34     "Boxhill_Doubles": {
35     "f1@0.25": 0.10526315789473685,
36     "f1@0.5": 0.03508771929824561,
37     "merge_count": 7,
38     "merge_rate": 0.9534883720930233,
39     "num_gt": 43,
40     "num_pred": 14,
41     "split_count": 0,
42     "tol_f1": 0.0
43     },
44     "GX010128": {
45     "f1@0.25": 0.14925373134328357,
46     "f1@0.5": 0.05970149253731343,
47     "merge_count": 12,
```

```
48         "merge_rate": 0.9615384615384616,
49         "num_gt": 52,
50         "num_pred": 15,
51         "split_count": 0,
52         "tol_f1": 0.0
53     },
54     "GX010137": {
55         "f1@0.25": 0.9253731343283583,
56         "f1@0.5": 0.5970149253731343,
57         "merge_count": 3,
58         "merge_rate": 0.17647058823529413,
59         "num_gt": 34,
60         "num_pred": 33,
61         "split_count": 0,
62         "tol_f1": 0.4477611940298507
63     },
64     "GX010141": {
65         "f1@0.25": 0.5416666666666666,
66         "f1@0.5": 0.3333333333333333,
67         "merge_count": 5,
68         "merge_rate": 0.6551724137931034,
69         "num_gt": 29,
70         "num_pred": 19,
71         "split_count": 0,
72         "tol_f1": 0.3333333333333333
73     },
74     "GX020094": {
75         "f1@0.25": 0.7333333333333334,
76         "f1@0.5": 0.4444444444444445,
77         "merge_count": 3,
78         "merge_rate": 0.15,
79         "num_gt": 40,
80         "num_pred": 50,
81         "split_count": 0,
82         "tol_f1": 0.2888888888888889
83     },
84     "GX030094": {
85         "f1@0.25": 0.5352112676056339,
86         "f1@0.5": 0.28169014084507044,
87         "merge_count": 8,
88         "merge_rate": 0.5609756097560976,
89         "num_gt": 41,
90         "num_pred": 30,
91         "split_count": 0,
92         "tol_f1": 0.14084507042253522
93     },
94     "adarsh_avi_singles": {
95         "f1@0.25": 0.8601036269430052,
96         "f1@0.5": 0.6321243523316062,
97         "merge_count": 5,
98         "merge_rate": 0.10416666666666667,
99         "num_gt": 96,
100        "num_pred": 97,
101        "split_count": 0,
102        "tol_f1": 0.38341968911917096
103    },
104    "gbaaddy": {
105        "f1@0.25": 0.17142857142857143,
106        "f1@0.5": 0.08571428571428572,
107        "merge_count": 9,
108        "merge_rate": 0.8541666666666666,
109        "num_gt": 48,
110        "num_pred": 22,
111        "split_count": 0,
112        "tol_f1": 0.0
```

```

113     },
114     "kushagra_singles": {
115         "f1@0.25": 0.0,
116         "f1@0.5": 0.0,
117         "merge_count": 2,
118         "merge_rate": 1.0,
119         "num_gt": 32,
120         "num_pred": 2,
121         "split_count": 0,
122         "tol_f1": 0.0
123     },
124     "largetest_doubles": {
125         "f1@0.25": 0.8118811881188118,
126         "f1@0.5": 0.5346534653465348,
127         "merge_count": 4,
128         "merge_rate": 0.16326530612244897,
129         "num_gt": 49,
130         "num_pred": 52,
131         "split_count": 0,
132         "tol_f1": 0.297029702970297
133     },
134     "mahadevpura_1": {
135         "f1@0.25": 0.0,
136         "f1@0.5": 0.0,
137         "merge_count": 1,
138         "merge_rate": 1.0,
139         "num_gt": 10,
140         "num_pred": 1,
141         "split_count": 0,
142         "tol_f1": 0.0
143     },
144     "mahadevpura_2": {
145         "f1@0.25": 0.0,
146         "f1@0.5": 0.0,
147         "merge_count": 3,
148         "merge_rate": 0.975,
149         "num_gt": 40,
150         "num_pred": 5,
151         "split_count": 0,
152         "tol_f1": 0.0
153     },
154     "mahadevpura_singles": {
155         "f1@0.25": 0.6181818181818182,
156         "f1@0.5": 0.3636363636363636,
157         "merge_count": 6,
158         "merge_rate": 0.5161290322580645,
159         "num_gt": 31,
160         "num_pred": 24,
161         "split_count": 0,
162         "tol_f1": 0.1818181818181818
163     },
164     "testlarge_short": {
165         "f1@0.25": 0.5454545454545454,
166         "f1@0.5": 0.1818181818181818,
167         "merge_count": 1,
168         "merge_rate": 0.5,
169         "num_gt": 6,
170         "num_pred": 5,
171         "split_count": 0,
172         "tol_f1": 0.0
173     }
174 },
175 "preset": {
176     "blob_factor": 4.0,
177     "blob_fallback": false,

```

```

178         "hard_cap": false,
179         "inactivity_end": 0.0,
180         "inactivity_min_density": 0.34,
181         "inactivity_rally_thresh": 0.08,
182         "inactivity_temporal_density": true,
183         "max_window_duration": 0.0,
184         "merge_gap": 2.0,
185         "min_window_duration": 2.0,
186         "name": "baseline_off",
187         "velocity_thresh": 0.02
188     }
189 },
190 "default": {
191     "aggregate": {
192         "dend_abs_median": 2.7189294850405976,
193         "dstart_abs_median": 3.1915265376487665,
194         "f1@0.25": 0.6351342900954481,
195         "f1@0.5": 0.47226205620233463,
196         "merge_rate": 0.13495907349355626,
197         "precision": 0.4144811747149071,
198         "recall": 0.5722200200034974,
199         "segment_count_ratio": 1.3752983331901796,
200         "split_rate": 0.1077567606184701,
201         "tol_f1": 0.36356483482541063,
202         "tol_precision": 0.3193624497645106,
203         "tol_recall": 0.44074854650051454
204     },
205     "iou": 0.5,
206     "min_crossings": 0,
207     "n": 15,
208     "note": "SERVED production windowing \u2014 now the R1 milestone (cap=6 + R4)
after the #204 flip.",
209     "per_video": {
210         "Badminton_BXH_2": {
211             "f1@0.25": 0.6557377049180326,
212             "f1@0.5": 0.459016393442623,
213             "merge_count": 2,
214             "merge_rate": 0.09090909090909091,
215             "num_gt": 44,
216             "num_pred": 78,
217             "split_count": 4,
218             "tol_f1": 0.3442622950819672
219         },
220         "Boxhill_Doubles": {
221             "f1@0.25": 0.5625,
222             "f1@0.5": 0.42187499999999994,
223             "merge_count": 0,
224             "merge_rate": 0.0,
225             "num_gt": 43,
226             "num_pred": 85,
227             "split_count": 8,
228             "tol_f1": 0.28125
229         },
230         "GX010128": {
231             "f1@0.25": 0.7164179104477613,
232             "f1@0.5": 0.582089552238806,
233             "merge_count": 0,
234             "merge_rate": 0.0,
235             "num_gt": 52,
236             "num_pred": 82,
237             "split_count": 2,
238             "tol_f1": 0.47761194029850756
239         },
240         "GX010137": {
241             "f1@0.25": 0.8571428571428571,

```

```

242         "f1@0.5": 0.7999999999999999,
243         "merge_count": 0,
244         "merge_rate": 0.0,
245         "num_gt": 34,
246         "num_pred": 36,
247         "split_count": 1,
248         "tol_f1": 0.6
249     },
250     "GX010141": {
251         "f1@0.25": 0.7843137254901961,
252         "f1@0.5": 0.5490196078431373,
253         "merge_count": 5,
254         "merge_rate": 0.3793103448275862,
255         "num_gt": 29,
256         "num_pred": 22,
257         "split_count": 0,
258         "tol_f1": 0.47058823529411764
259     },
260     "GX020094": {
261         "f1@0.25": 0.6966292134831462,
262         "f1@0.5": 0.4719101123595506,
263         "merge_count": 0,
264         "merge_rate": 0.0,
265         "num_gt": 40,
266         "num_pred": 49,
267         "split_count": 6,
268         "tol_f1": 0.4044943820224719
269     },
270     "GX030094": {
271         "f1@0.25": 0.6206896551724138,
272         "f1@0.5": 0.46551724137931033,
273         "merge_count": 0,
274         "merge_rate": 0.0,
275         "num_gt": 41,
276         "num_pred": 75,
277         "split_count": 6,
278         "tol_f1": 0.3620689655172414
279     },
280     "adarsh_avi_singles": {
281         "f1@0.25": 0.7962962962962963,
282         "f1@0.5": 0.6944444444444444,
283         "merge_count": 0,
284         "merge_rate": 0.0,
285         "num_gt": 96,
286         "num_pred": 120,
287         "split_count": 12,
288         "tol_f1": 0.5740740740740741
289     },
290     "gbaaddy": {
291         "f1@0.25": 0.6567164179104478,
292         "f1@0.5": 0.44776119402985076,
293         "merge_count": 1,
294         "merge_rate": 0.041666666666666664,
295         "num_gt": 48,
296         "num_pred": 86,
297         "split_count": 7,
298         "tol_f1": 0.28358208955223885
299     },
300     "kushagra_singles": {
301         "f1@0.25": 0.6933333333333334,
302         "f1@0.5": 0.45333333333333325,
303         "merge_count": 3,
304         "merge_rate": 0.1875,
305         "num_gt": 32,
306         "num_pred": 43,

```

```

307         "split_count": 6,
308         "tol_f1": 0.10666666666666666
309     },
310     "targettest_doubles": {
311         "f1@0.25": 0.7927927927927928,
312         "f1@0.5": 0.6306306306306306,
313         "merge_count": 0,
314         "merge_rate": 0.0,
315         "num_gt": 49,
316         "num_pred": 62,
317         "split_count": 4,
318         "tol_f1": 0.5405405405405406
319     },
320     "mahadevpura_1": {
321         "f1@0.25": 0.0,
322         "f1@0.5": 0.0,
323         "merge_count": 2,
324         "merge_rate": 0.8,
325         "num_gt": 10,
326         "num_pred": 2,
327         "split_count": 0,
328         "tol_f1": 0.0
329     },
330     "mahadevpura_2": {
331         "f1@0.25": 0.525,
332         "f1@0.5": 0.25,
333         "merge_count": 9,
334         "merge_rate": 0.525,
335         "num_gt": 40,
336         "num_pred": 40,
337         "split_count": 3,
338         "tol_f1": 0.15
339     },
340     "mahadevpura_singles": {
341         "f1@0.25": 0.7249999999999999,
342         "f1@0.5": 0.525,
343         "merge_count": 0,
344         "merge_rate": 0.0,
345         "num_gt": 31,
346         "num_pred": 49,
347         "split_count": 6,
348         "tol_f1": 0.525
349     },
350     "testlarge_short": {
351         "f1@0.25": 0.4444444444444444,
352         "f1@0.5": 0.3333333333333333,
353         "merge_count": 0,
354         "merge_rate": 0.0,
355         "num_gt": 6,
356         "num_pred": 12,
357         "split_count": 1,
358         "tol_f1": 0.3333333333333333
359     }
360 },
361 "preset": {
362     "blob_factor": 4.0,
363     "blob_fallback": false,
364     "hard_cap": false,
365     "inactivity_end": 1.5,
366     "inactivity_min_density": 0.3,
367     "inactivity_rally_thresh": 0.2,
368     "inactivity_temporal_density": true,
369     "max_window_duration": 6.0,
370     "merge_gap": 2.0,
371     "min_window_duration": 2.0,

```

```

372         "name": "served",
373         "velocity_thresh": 0.02
374     }
375 },
376 "milestone": {
377     "aggregate": {
378         "dend_abs_median": 2.7189294850405976,
379         "dstart_abs_median": 3.1915265376487665,
380         "f1@0.25": 0.6351342900954481,
381         "f1@0.5": 0.47226205620233463,
382         "merge_rate": 0.13495907349355626,
383         "precision": 0.4144811747149071,
384         "recall": 0.5722200200034974,
385         "segment_count_ratio": 1.3752983331901796,
386         "split_rate": 0.1077567606184701,
387         "tol_f1": 0.36356483482541063,
388         "tol_precision": 0.3193624497645106,
389         "tol_recall": 0.44074854650051454
390     },
391     "iou": 0.5,
392     "min_crossings": 0,
393     "n": 15,
394     "note": "R3 cap=6 s + R4 rally-END inactivity trim \u2014 the shipped milestone
(== default post-#204).",
395     "per_video": {
396         "Badminton_BXH_2": {
397             "f1@0.25": 0.6557377049180326,
398             "f1@0.5": 0.459016393442623,
399             "merge_count": 2,
400             "merge_rate": 0.09090909090909091,
401             "num_gt": 44,
402             "num_pred": 78,
403             "split_count": 4,
404             "tol_f1": 0.3442622950819672
405         },
406         "Boxhill_Doubles": {
407             "f1@0.25": 0.5625,
408             "f1@0.5": 0.42187499999999994,
409             "merge_count": 0,
410             "merge_rate": 0.0,
411             "num_gt": 43,
412             "num_pred": 85,
413             "split_count": 8,
414             "tol_f1": 0.28125
415         },
416         "GX010128": {
417             "f1@0.25": 0.7164179104477613,
418             "f1@0.5": 0.582089552238806,
419             "merge_count": 0,
420             "merge_rate": 0.0,
421             "num_gt": 52,
422             "num_pred": 82,
423             "split_count": 2,
424             "tol_f1": 0.47761194029850756
425         },
426         "GX010137": {
427             "f1@0.25": 0.8571428571428571,
428             "f1@0.5": 0.7999999999999999,
429             "merge_count": 0,
430             "merge_rate": 0.0,
431             "num_gt": 34,
432             "num_pred": 36,
433             "split_count": 1,
434             "tol_f1": 0.6
435         },

```

```
436 "GX010141": {
437   "f1@0.25": 0.7843137254901961,
438   "f1@0.5": 0.5490196078431373,
439   "merge_count": 5,
440   "merge_rate": 0.3793103448275862,
441   "num_gt": 29,
442   "num_pred": 22,
443   "split_count": 0,
444   "tol_f1": 0.47058823529411764
445 },
446 "GX020094": {
447   "f1@0.25": 0.6966292134831462,
448   "f1@0.5": 0.4719101123595506,
449   "merge_count": 0,
450   "merge_rate": 0.0,
451   "num_gt": 40,
452   "num_pred": 49,
453   "split_count": 6,
454   "tol_f1": 0.4044943820224719
455 },
456 "GX030094": {
457   "f1@0.25": 0.6206896551724138,
458   "f1@0.5": 0.46551724137931033,
459   "merge_count": 0,
460   "merge_rate": 0.0,
461   "num_gt": 41,
462   "num_pred": 75,
463   "split_count": 6,
464   "tol_f1": 0.3620689655172414
465 },
466 "adarsh_avi_singles": {
467   "f1@0.25": 0.7962962962962963,
468   "f1@0.5": 0.6944444444444444,
469   "merge_count": 0,
470   "merge_rate": 0.0,
471   "num_gt": 96,
472   "num_pred": 120,
473   "split_count": 12,
474   "tol_f1": 0.5740740740740741
475 },
476 "gbaaddy": {
477   "f1@0.25": 0.6567164179104478,
478   "f1@0.5": 0.44776119402985076,
479   "merge_count": 1,
480   "merge_rate": 0.041666666666666664,
481   "num_gt": 48,
482   "num_pred": 86,
483   "split_count": 7,
484   "tol_f1": 0.28358208955223885
485 },
486 "kushagra_singles": {
487   "f1@0.25": 0.6933333333333334,
488   "f1@0.5": 0.45333333333333325,
489   "merge_count": 3,
490   "merge_rate": 0.1875,
491   "num_gt": 32,
492   "num_pred": 43,
493   "split_count": 6,
494   "tol_f1": 0.10666666666666666
495 },
496 "targetest_doubles": {
497   "f1@0.25": 0.7927927927927928,
498   "f1@0.5": 0.6306306306306306,
499   "merge_count": 0,
500   "merge_rate": 0.0,
```

```

501         "num_gt": 49,
502         "num_pred": 62,
503         "split_count": 4,
504         "tol_f1": 0.5405405405405406
505     },
506     "mahadevpura_1": {
507         "f1@0.25": 0.0,
508         "f1@0.5": 0.0,
509         "merge_count": 2,
510         "merge_rate": 0.8,
511         "num_gt": 10,
512         "num_pred": 2,
513         "split_count": 0,
514         "tol_f1": 0.0
515     },
516     "mahadevpura_2": {
517         "f1@0.25": 0.525,
518         "f1@0.5": 0.25,
519         "merge_count": 9,
520         "merge_rate": 0.525,
521         "num_gt": 40,
522         "num_pred": 40,
523         "split_count": 3,
524         "tol_f1": 0.15
525     },
526     "mahadevpura_singles": {
527         "f1@0.25": 0.7249999999999999,
528         "f1@0.5": 0.525,
529         "merge_count": 0,
530         "merge_rate": 0.0,
531         "num_gt": 31,
532         "num_pred": 49,
533         "split_count": 6,
534         "tol_f1": 0.525
535     },
536     "testlarge_short": {
537         "f1@0.25": 0.4444444444444444,
538         "f1@0.5": 0.3333333333333333,
539         "merge_count": 0,
540         "merge_rate": 0.0,
541         "num_gt": 6,
542         "num_pred": 12,
543         "split_count": 1,
544         "tol_f1": 0.3333333333333333
545     }
546 },
547 "preset": {
548     "blob_factor": 4.0,
549     "blob_fallback": false,
550     "hard_cap": false,
551     "inactivity_end": 1.5,
552     "inactivity_min_density": 0.3,
553     "inactivity_rally_thresh": 0.2,
554     "inactivity_temporal_density": true,
555     "max_window_duration": 6.0,
556     "merge_gap": 2.0,
557     "min_window_duration": 2.0,
558     "name": "milestone",
559     "velocity_thresh": 0.02
560 }
561 }
562 },
563 "generated_at": "2026-07-03",
564 "git_commit": "a68f02e",
565 "iou": 0.5,

```

```

566     "manifest": "output/human_lovo_manifest.json",
567     "tolerances": {
568         "metric_tol": 0.005,
569         "over_seg_tol": 0,
570         "rate_tol": 0.01
571     }
572 }
573

```

8. assistant (2026-07-06T21:59:58.593Z)

I have the exact preset values already from the baseline JSON. The `default` and `milestone` presets are identical, with a `preset` block embedding all the knobs. Now let me read the `trajectory\_to\_action\_windows` function to get its full signature and confirm each knob maps to a parameter.

9. user (2026-07-06T22:00:00.099Z)

```

311     def trajectory_to_action_windows(
312         points: List[TrajectoryPoint],
313         fps: float,
314         frame_width: float,
315         chunk_start: float = 0.0,
316         velocity_thresh: float = 0.02,
317         merge_gap: float = 2.0,
318         min_window_duration: float = 2.0,
319         chunk_index: Optional[int] = None,
320         max_window_duration: float = 0.0,
321         min_split_gap: float = 0.5,
322         window_hard_cap: bool = False,
323         window_inactivity_end: float = 0.0,
324         inactivity_min_density: float = 0.34,
325         inactivity_rally_thresh: float = 0.08,
326         inactivity_temporal_density: bool = False,
327         window_blob_fallback: bool = False,
328         window_blob_factor: float = 4.0,
329     ) -> List[Dict[str, Any]]:
330         """Convert a shuttle trajectory into candidate rally windows.
331
332         A frame is "active" when the shuttle is visible AND its inter-frame
333         displacement (normalised by frame width, per second) exceeds
334         ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
335         ``merge_gap`` seconds of each other are grouped into one window; short
336         windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
337         shape feeding the AI-handoff phase is identical.
338
339         ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
340         against the *over-merge* failure where "shuttle moving" is conflated with "rally in
341         progress" and a single window swallows several rallies + the dead time between them. When
342         set, any window longer than this is recursively SPLIT at its largest internal inactivity
343         gap (the longest stretch with no in-flight shuttle ? the most likely rally boundary),
344         provided that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing
345         fix from ``docs/RALLY_QUALITY_RESEARCH.md`` §6 (W1); it never merges, only cuts, so
346         recall cannot drop, and it is config-gated default-OFF until measured on the golden set.
347
348         ``window_hard_cap`` (default OFF) makes ``max_window_duration`` a TRUE cap: when
349         an over-long window has NO internal inactivity gap to split on (a continuously-

```

```

active
350         trajectory ? the shuttle never stops moving, e.g. the real-footage
``mahadevpura-1``
351         blob where the gap-splitter alone left a single 174 s window), it is recursively
352         hard-chopped at its temporal midpoint until every piece is ? the cap. Still only
cuts
353         (never merges), so recall cannot drop; gated default-OFF until measured.
354
355         ``window_blob_fallback`` (default OFF ? the R5 last-resort blob splitter):
unlike
356         ``window_hard_cap`` (which midpoint-chops BEFORE the R4 trim, so it fires on
every gap-less
357         over-cap window and over-segments normal clips), this runs AFTER the R4
inactivity trim and
358         force-chops ONLY a genuine *blob* ? a group still longer than
``window_blob_factor`` *
359         ``max_window_duration`` once the gap-split and the principled rally-end trim
have both had
360         their chance (e.g. mahadevpura-1's ~65 s residual ? 11× a 6 s cap; a 7 s rally
is left
361         alone). The blob is midpoint-chopped down to ? the cap, those forced cuts are
logged, and
362         each resulting window is flagged ``forced_cut=True`` ? surfacing the clip as one
that needs
363         rally-STATE cues (net-crossings / two-sided motion), not a bigger cap. Only cuts
(recall
364         can't drop). Requires ``max_window_duration`` > 0; gated default-OFF until
measured.
365
366         if fps <= 0:
367             fps = 30.0
368         if frame_width <= 0:
369             frame_width = 1280.0
370
371         active = TrackNetRunner._active_frames(
372             points, fps, frame_width, velocity_thresh
373         )
374
375         if not active:
376             return []
377
378         max_gap_frames = int(merge_gap * fps)
379
380         def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
381             vels = [v for _, v in group]
382             return {
383                 "start": group[0][0] / fps + chunk_start,
384                 "end": group[-1][0] / fps + chunk_start,
385                 "active_frame_count": len(group),
386                 "peak_velocity": max(vels),
387                 "mean_velocity": sum(vels) / len(vels),
388                 "track_id_count": 1, # single shuttle; kept for window-shape parity
389                 "chunk_index": chunk_index,
390             }
391
392         groups: List[List[Tuple[int, float]]] = []
393         group = [active[0]]
394         for af in active[1:]:
395             if af[0] - group[-1][0] <= max_gap_frames:
396                 group.append(af)
397             else:
398                 groups.append(group)
399                 group = [af]
400         groups.append(group)
401

```

```

402         if max_window_duration > 0:
403             groups = TrackNetRunner._split_overlong_groups(
404                 groups,
405                 int(max_window_duration * fps),
406                 int(min_split_gap * fps),
407                 hard_cap=window_hard_cap,
408             )
409
410         # R4 ? rally-END inactivity trim (default OFF). Applied AFTER the cap split so
each rally
411         # piece has its own post-rally drift tail removed: the velocity windowing keeps
a window
412         # "active" while the shuttle drifts/gets-picked-up above threshold, over-
extending the END
413         # (the measured gap vs golden). Trim back to the last frame whose trailing
window is still
414         # dense with motion (rally on); a sustained low-density run = rally over. Only
cuts.
415         if window_inactivity_end > 0:
416             tw = int(window_inactivity_end * fps)
417             groups = [
418                 TrackNetRunner._trim_inactive_tail(
419                     g,
420                     tw,
421                     inactivity_min_density,
422                     inactivity_rally_thresh,
423                     temporal_density=inactivity_temporal_density,
424                 )
425                 for g in groups
426             ]
427
428         # R5 ? blob-fallback (default OFF). LAST resort: after the gap-split AND the R4
trim, any
429         # group still longer than the cap is a gap-less, rally-end-signal-less blob;
force-chop it to
430         # ? the cap at the midpoint and flag every piece, so the degenerate single-
window outcome is
431         # avoided and the clip is surfaced (logged) as needing rally-STATE cues, not a
bigger cap.
432         forced_flags = [False] * len(groups)
433         if window_blob_fallback and max_window_duration > 0:
434             groups, forced_flags = TrackNetRunner._blob_fallback_chop(
435                 groups, int(max_window_duration * fps), window_blob_factor
436             )
437
438         windows = []
439         for g, forced in zip(groups, forced_flags):
440             w = _finalize(g)
441             if forced:
442                 w["forced_cut"] = True
443             windows.append(w)
444         return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
445
446     @staticmethod
447     def _trim_inactive_tail(
448         group: List[Tuple[int, float]],
449         trail_frames: int,
450         min_density: float,
451         rally_thresh: float,
452         temporal_density: bool = False,
453     ) -> List[Tuple[int, float]]:
454         """R4: trim the post-rally drift tail. After a rally the shuttle keeps moving
*slowly*
455         (picked up / walked / knock-up) above ``velocity_thresh``, so the velocity
windowing

```

456 over-extends the END. A frame is "fast" (real rally motion) iff its velocity >
``rally\_thresh``
457 (higher than the active threshold). The rally is "on" at frame ``i`` while its
trailing
458 ``trail\_frames`` window holds >= ``min\_density`` fraction of fast frames; trim
back to the
459 LAST such frame ? everything after is sustained slow drift (rally over). Only
cuts (recall
460 can't rise); a window whose tail never goes slow is untouched, and if no fast-
dense point

10. assistant (2026-07-06T22:00:05.484Z)

I have the full signature. Now I need to map the baseline JSON's `preset` keys to the function's parameter names. The preset uses keys like `hard\_cap`, `inactivity\_end`, `blob\_factor`, `blob\_fallback` which differ from the function param names. Let me find where the preset dict is defined and how it maps to the function call.

11. user (2026-07-06T22:00:07.612Z)

Found 2 files
backend\eval\rally_seg_eval.py
backend\eval\windowing.py

12. user (2026-07-06T22:00:09.133Z)

Found 29 files
tests\test_config_deployment.py
backend\config\models.py
deploy\cloudrun\gen_service_yaml.py
tests\test_startup_checks.py
tests\test_native_wasb_runner.py
tests\test_config_wasb.py
backend\pipeline\detectors\native_wasb_runner.py
backend\eval\distill_local.py
backend\eval\rally_seg_eval.py
backend\eval\serve_contrast.py
backend\config\presets.py
backend\config\startup.py
backend\pipeline\stitcher\compiler.py
backend\utils\video.py
backend\utils\annotation_proxy.py
tests\test_windowing.py
tests\test_rally_seg_eval.py
backend\eval\windowing.py
tests\test_serve_contrast.py
tests\test_promotion.py
tests\test_rally_seg_eval_report.py
tests\test_nightly_regression.py
tests\test_p2b_install.py
docs\archives\quality-iterations\2026-06-18-lovo-resweep-n13\lovo_resweep.py
scripts\nightly_regression.py
backend\tools\rally_slicer.py
backend\eval\promotion.py
backend\config\loader.py
backend\config__init__.py

13. assistant (2026-07-06T22:00:17.010Z)

There's a `backend/config/presets.py` and `backend/eval/windowing.py`. These are the most likely places the preset is defined and mapped. Let me read both.

14. user (2026-07-06T22:00:17.313Z)

```

1      """Deployment-profile presets + environment auto-detection (COMPUTE_DECOUPLED_SERVING
M1).
2
3      A *deployment profile* is a single switch that applies a coherent BUNDLE of existing
4      config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one
5      knob at a time (the design in ``docs/COMPUTE_DECOUPLED_SERVING/README.md`` §4.3).
6
7      Precedence (lowest ? highest), realised by the loader::
8
9          built-in model defaults < profile preset < config.json < env < worker --set
10
11      This module is **pure data + pure functions** ? no IO, and (critically) **no torch
import at
12      module load**. The loader applies a preset ONLY when a profile is EXPLICITLY selected
13      (``config.json`` ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); with no
14      profile selected the loader is byte-identical to the pre-M1 behaviour (``profile=""`` ==
15      today). Auto-detect only ever **suggests** a profile ? it never auto-applies one (D15:
cloud
16      spend is never entered silently).
17      """
18
19      from __future__ import annotations
20
21      import copy
22      import os
23      from typing import Any, Mapping, Optional
24
25      # Canonical enum values, defined ONCE here. ``models.DeploymentConfig``'s ``Literal``
mirrors
26      # these and ``tests/test_config_deployment.py`` pins parity so the two can never drift.
27      PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
28      COMPUTE_TARGETS: tuple[str, ...] = ("local_gpu", "cloud_run", "cpu_mock")
29
30      # Each profile maps 1:1 to its canonical compute_target. Used to apply the preset and to
warn
31      # when config.json explicitly overrides compute_target into a state inconsistent with
the profile.
32      COMPUTE_TARGET_FOR_PROFILE: dict[str, str] = {
33          "truly-local": "local_gpu",
34          "cloud-serving": "cloud_run",
35          "cpu-mock": "cpu_mock",
36      }
37
38      # A preset is a partial, nested override dict over the model defaults. It touches ONLY
knobs
39      # that EXIST today ? M1 is pure/additive: the new
``deployment.{profile,compute_target}`` plus
40      # ``indexing.{detector_impl,skip_ai_handoff}`` and ``storage.backend``. Later milestones
extend
41      # these bundles ? ``input_backend`` (M2) and the configurable output/workspace base (M3,
42      # Launch-Report-gated) ? deliberately NOT set here so M1 never pre-empts a gated flip.
43      PROFILE_PRESETS: dict[str, dict[str, Any]] = {
44          "truly-local": {
45              "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
46              # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU
box).
47              # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
48              "indexing": {"skip_ai_handoff": True},
49              "storage": {"backend": "local"},
50          },
51          "cloud-serving": {
52              "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
53              # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor
(D11).
54              # wasb GPU-feed tuning: the parity defaults (batch 8, no prefetch) left the paid

```

```

L4
55         # <1% utilized on the first real serving run (2026-07-03) ? the pipeline was
56         # CPU-feed-bound. Serving opts into the measured fast values; local/WSL
unchanged.
57         "indexing": {
58             "detector_impl": "native",
59             "skip_ai_handoff": False,
60             "wasb": {"batch_size": 64, "prefetch_batches": 4},
61         },
62         "storage": {"backend": "gcs"},
63     },
64     "cpu-mock": {
65         "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
66         # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression
profile.
67         "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
68         "storage": {"backend": "local"},
69     },
70 }
71
72
73 def is_known_profile(profile: str) -> bool:
74     """True for a non-empty, recognised profile name."""
75     return bool(profile) and profile in PROFILE_PRESETS
76
77
78 def preset_for(profile: str) -> dict[str, Any]:
79     """Return a deep COPY of the preset bundle for ``profile`` (so callers can't mutate
the
80     module-level table), or ``{}`` for ``""`` / an unknown profile."""
81     return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))
82
83
84 def _truthy(val: Optional[str]) -> bool:
85     """Common env-var truthiness (``1/true/yes/on``, case/space-insensitive)."""
86     if val is None:
87         return False
88     return val.strip().lower() in {"1", "true", "yes", "on"}
89
90
91 def _cuda_available() -> bool:
92     """Best-effort CUDA probe. torch is heavy + optional, so it is imported lazily here
and
93     ONLY when a caller opts into ``probe_cuda``; any import/runtime failure means 'no
GPU'."""
94     try:
95         # Intentionally lazy + local: keeps the config-load path torch-free (torch is
heavy
96         # and optional). Only reached when a caller opts into probe_cuda.
97         import torch
98
99         return bool(torch.cuda.is_available())
100     except Exception:
101         return False
102
103
104 def probe_cuda_available() -> bool:
105     """Public best-effort CUDA probe (lazy torch import; ``False`` on any failure).
106
107     Exposed for the per-profile startup checks (``backend/config/startup.py``) so a
caller
108     that *wants* the GPU heads-up can pass the result in without the startup module
importing
109     torch itself. Best-effort by design: if torch lives only in a detector subprocess
env

```

```

110         (native WASB), this returns ``False`` in the main process ? which is why the startup
checks
111         treat a GPU mismatch as a WARNING (the runner healthcheck is the authority), never a
hard
112         error that could false-fail."""
113         return _cuda_available()
114
115
116     def suggest_profile(
117         env: Optional[Mapping[str, str]] = None, *, probe_cuda: bool = False
118     ) -> str:
119         """SUGGEST (never auto-apply ? D15) a deployment profile from the environment.
120
121         Signals, in priority order:
122
123         - ``K_SERVICE`` set (Cloud Run)           ? ``"cloud-serving"``
124         - ``CI`` truthy                            ? ``"cpu-mock"``
125         - ``probe_cuda`` and a CUDA GPU present ? ``"truly-local"``
126         - otherwise                                ? ``""`` (no strong suggestion)
127
128         ``probe_cuda`` defaults to ``False`` so the common config-load path stays cheap and
never
129         imports torch; a CLI ``--detect`` or cloud-serving startup that wants the GPU probe
passes
130         ``probe_cuda=True``. The result is advisory only: the caller decides whether to act
on it.
131         """
132         e = os.environ if env is None else env
133         if e.get("K_SERVICE"):
134             return "cloud-serving"
135         if _truthy(e.get("CI")):
136             return "cpu-mock"
137         if probe_cuda and _cuda_available():
138             return "truly-local"
139         return ""
140

```

15. user (2026-07-06T22:00:17.378Z)

```

1     """Single-sourced candidate-windowing presets ? the eval?serve foundation fix.
2
3     **The bug this closes.** The offline eval substrate (``distill_local.build_candidates``
?
4     ``fusion_golden`` ? the golden feature JSONs the ablation/compare path re-scores) was
built
5     on a *recall-oriented PROPOSAL* windowing (``vt=0.005, merge_gap=1.0, min_window=0.5``):
6     deliberately permissive so a learned scorer/gate can filter the junk. But the SERVED
pipeline
7     (``TrajectoryHybridSegmenter.process_video``) windows COARSELY (``vt=0.02,
merge_gap=2.0,
8     min_window=2.0`` ? the ``config.indexing`` defaults). A cue measured on the proposal set
is
9     measured on candidates **production never serves**, so a "win" need not transfer. This
is
10    exactly how Gate-A scored **~0.074** on the proposal set yet **~0.008** on the served
set
11    (``scratch/GATE_AB_NUMBERS_AND_IMPLICATIONS.md`` vs ``output/gateA_served_verify/``).
12
13    **The fix.** One module owns the two named windowings as :class:`WindowingPreset`s, and
the
14    SERVED preset is **single-sourced from the same Pydantic config defaults production
reads**
15    (:class:`backend.config.models.IndexingConfig`) ? eval and serve can no longer drift.
Helpers
16    build candidates under either preset, and :func:`served_windowing_from_config` lifts the

```

```

17     *actual* served windowing out of a live config (overrides included), so an A/B can
target the
18     operating point a given deployment truly serves.
19
20     The companion rule ? *a cue is promotable only if it does NOT regress at the SERVED
21     windowing* ? lives in :mod:`backend.eval.promotion` (which composes ``eval_stats`` +
22     ``per_user_gate``). This module is the what windowing; that one is the promotion
gate**.
23
24     Pure, offline, stdlib + config only. Nothing here changes existing eval behaviour until
a
25     caller opts in (``distill_local`` re-points its module-level presets here, byte-
identically).
26     """
27
28     from __future__ import annotations
29
30     from dataclasses import dataclass
31     from typing import Any, Dict, List, Mapping, Tuple
32
33     from backend.config.models import IndexingConfig
34
35     Interval = Tuple[float, float]
36
37     __all__ = [
38         "WindowingPreset",
39         "SERVED",
40         "PROPOSAL",
41         "PRESETS",
42         "served_windowing_from_config",
43         "build_windows",
44         "windows_to_intervals",
45     ]
46
47
48     @dataclass(frozen=True)
49     class WindowingPreset:
50         """A named (velocity_thresh, merge_gap, min_window_duration) windowing operating
point.
51
52         ``velocity_thresh`` ? normalised per-second shuttle velocity above which a frame
counts as
53         "in flight"; ``merge_gap`` ? seconds within which active frames coalesce into one
window;
54         ``min_window_duration`` ? windows shorter than this are dropped. These are exactly
the three
55         knobs ``TrackNetRunner.trajectory_to_action_windows`` takes, so a preset is the
windowing.
56         """
57
58         name: str
59         velocity_thresh: float
60         merge_gap: float
61         min_window_duration: float
62         #: 0 = OFF (default). When > 0, windows longer than this are split at their largest
internal
63         #: inactivity gap ? the bounded-windowing over-merge fix (W1,
RALLY_QUALITY_RESEARCH.md §6).
64         max_window_duration: float = 0.0
65         #: When True, ``max_window_duration`` is a HARD cap ? an over-long window with no
internal
66         #: inactivity gap is chopped at its midpoint until ? the cap (continuously-active
fix). OFF by default.
67         hard_cap: bool = False
68         #: R4 rally-END inactivity trim (s, 0 = OFF): trim a window's post-rally slow-drift

```

```

tail back to
69         #: the last frame whose trailing window stays dense with FAST motion. The measured
rally-end fix.
70         inactivity_end: float = 0.0
71         #: velocity above which motion counts as "rally" (vs slow drift) for the R4 trim;
min fast-fraction.
72         inactivity_rally_thresh: float = 0.08
73         inactivity_min_density: float = 0.34
74         #: R4 density-denominator fix (finding B), default OFF. When True the R4 fast-
fraction is taken
75         #: over the trailing window's TEMPORAL length, not the count of active samples
(which sparse
76         #: tracking inflates). No-op under dense tracking; gated default-OFF until measured
on SERVED.
77         inactivity_temporal_density: bool = False
78         #: R5 blob-fallback (default OFF). After the cap split + R4 trim, midpoint-chop any
group still
79         #: longer than ``blob_factor`` × ``max_window_duration`` (the gap-less, R4-survived
blob ?
80         #: mahadevpura-1's ~11× residual) and flag it. Runs AFTER R4 (vs ``hard_cap``
before) and the
81         #: factor keeps it surgical (a normal long rally is left alone).
82         blob_fallback: bool = False
83         blob_factor: float = 4.0
84
85         def as_tuple(self) -> Tuple[float, float, float]:
86             """``(velocity_thresh, merge_gap, min_window_duration)`` ? the legacy tuple
shape the
87             ``trajectory_to_action_windows`` / ``distill_local`` call sites already speak.
(The
88             bounded-windowing knob ``max_window_duration`` is passed separately by
``build_windows``.)"""
89             return (self.velocity_thresh, self.merge_gap, self.min_window_duration)
90
91         def to_dict(self) -> Dict[str, Any]:
92             return {
93                 "name": self.name,
94                 "velocity_thresh": self.velocity_thresh,
95                 "merge_gap": self.merge_gap,
96                 "min_window_duration": self.min_window_duration,
97                 "max_window_duration": self.max_window_duration,
98                 "hard_cap": self.hard_cap,
99                 "inactivity_end": self.inactivity_end,
100                 "inactivity_rally_thresh": self.inactivity_rally_thresh,
101                 "inactivity_min_density": self.inactivity_min_density,
102                 "inactivity_temporal_density": self.inactivity_temporal_density,
103                 "blob_fallback": self.blob_fallback,
104                 "blob_factor": self.blob_factor,
105             }
106
107
108         def _served_preset_from_defaults() -> WindowingPreset:
109             """The SERVED windowing, lifted from the SAME Pydantic config defaults production
reads.
110
111             ``TrajectoryHybridSegmenter.process_video`` reads exactly these three:
112             - velocity: ``idx_cfg[<family>].velocity_threshold`` (default 0.02 ?
wasb/fusion/tracknet)
113             - merge_gap: ``idx_cfg.dead_time_merge_gap`` (default 2.0)
114             - min window: ``idx_cfg.min_action_window_duration`` (default 2.0)
115
116             Sourcing them from :class:`IndexingConfig`'s defaults (not a hand-typed tuple) is
the whole
117             point: change a served default in ``config/models.py`` and the eval SERVED preset
follows,

```

```

118     so the two can never silently diverge again.
119     """
120     idx = IndexingConfig() # construct with defaults ? the served operating point
121     return WindowingPreset(
122         name="served",
123         velocity_thresh=float(
124             idx.wasb.velocity_threshold
125         ), # == fusion/tracknet default 0.02
126         merge_gap=float(idx.dead_time_merge_gap),
127         min_window_duration=float(idx.min_action_window_duration),
128         max_window_duration=float(
129             idx.max_window_duration
130         ), # W1 bounded-windowing (default 0=OFF)
131         hard_cap=bool(idx.window_hard_cap), # W1 hard-cap fallback (default OFF)
132         inactivity_end=float(
133             idx.window_inactivity_end
134         ), # R4 rally-end trim (default 0=OFF)
135         inactivity_rally_thresh=float(idx.inactivity_rally_thresh),
136         inactivity_min_density=float(idx.inactivity_min_density),
137         inactivity_temporal_density=bool(
138             idx.inactivity_temporal_density
139         ), # R4 density fix (finding B, default OFF)
140         blob_fallback=bool(idx.window_blob_fallback), # R5 blob-fallback (default OFF)
141         blob_factor=float(idx.window_blob_factor),
142     )
143
144
145     #: The COARSE windowing production actually serves (config defaults; ~0.02/2.0/2.0). A
cue is
146     #: only promotable if it holds up HERE ? the operating point real users get.
147     SERVED: WindowingPreset = _served_preset_from_defaults()
148
149     #: The recall-oriented PROPOSAL windowing the offline substrate is built on
(deliberately
150     #: permissive: propose many, let the classifier/gate filter). NOT what production serves
? a
151     #: win here must be re-confirmed on SERVED before promotion. Mirrors the historical
152     #: ``distill_local.PROPOSAL`` tuple (0.005/1.0/0.5) so the existing golden JSONs stay
valid.
153     PROPOSAL: WindowingPreset = WindowingPreset(
154         name="proposal", velocity_thresh=0.005, merge_gap=1.0, min_window_duration=0.5
155     )
156
157     #: Name ? preset, so a CLI / config string ("served" | "proposal") selects one.
158     PRESETS: Dict[str, WindowingPreset] = {SERVED.name: SERVED, PROPOSAL.name: PROPOSAL}
159
160
161     def served_windowing_from_config(config: Any, family: str = "wasb") -> WindowingPreset:
162         """The SERVED windowing a *specific* live config serves (overrides honoured).
163
164         Reproduces ``TrajectoryHybridSegmenter.process_video``'s knob reads exactly, against
an
165         arbitrary config (the typed :class:`AppConfig` or a plain dict) and segmenter
``family``
166         (``"wasb"`` / ``"fusion"`` / ``"tracknet"``). Use this when a deployment overrode a
served
167         default in ``config.json`` ? the module-level :data:`SERVED` is the *default*
operating point;
168         this is *this config's* operating point. Falls back to the served defaults for any
absent key.
169         """
170         idx = _as_mapping(config, "indexing")
171         family_cfg = _as_mapping(idx, family)
172         return WindowingPreset(
173             name=f"served:{family}",

```

```

174         velocity_thresh=float(
175             family_cfg.get("velocity_threshold", SERVED.velocity_thresh)
176         ),
177         merge_gap=float(idx.get("dead_time_merge_gap", SERVED.merge_gap)),
178         min_window_duration=float(
179             idx.get("min_action_window_duration", SERVED.min_window_duration)
180         ),
181         max_window_duration=float(
182             idx.get("max_window_duration", SERVED.max_window_duration)
183         ),
184         hard_cap=bool(idx.get("window_hard_cap", SERVED.hard_cap)),
185         inactivity_end=float(idx.get("window_inactivity_end", SERVED.inactivity_end)),
186         inactivity_rally_thresh=float(
187             idx.get("inactivity_rally_thresh", SERVED.inactivity_rally_thresh)
188         ),
189         inactivity_min_density=float(
190             idx.get("inactivity_min_density", SERVED.inactivity_min_density)
191         ),
192         inactivity_temporal_density=bool(
193             idx.get("inactivity_temporal_density", SERVED.inactivity_temporal_density)
194         ),
195         blob_fallback=bool(idx.get("window_blob_fallback", SERVED.blob_fallback)),
196         blob_factor=float(idx.get("window_blob_factor", SERVED.blob_factor)),
197     )
198
199
200     def _as_mapping(obj: Any, key: str) -> Mapping[str, Any]:
201         """`obj[key]` as a plain mapping, whether `obj` is a dict or a typed config
202         section
203         (`AppConfig.get` returns nested sections as dicts). Empty mapping if absent/non-
204         mapping."""
205         if obj is None:
206             return {}
207         if isinstance(obj, Mapping):
208             sub = obj.get(key, {})
209         elif hasattr(obj, "get"):
210             sub = obj.get(key, {})
211         else:
212             sub = getattr(obj, key, {})
213         return sub if isinstance(sub, Mapping) else {}
214
215     def build_windows(
216         points: List[Any], fps: float, frame_width: float, preset: WindowingPreset
217     ) -> List[Dict[str, Any]]:
218         """Trajectory `points` ? candidate windows under `preset` (the one windowing
219         call site).
220
221         A thin wrapper over `TrackNetRunner.trajectory_to_action_windows` that takes a
222         named preset
223         instead of three loose floats, so eval candidate-building and the served path
224         reference the
225         SAME knobs by name. Imported lazily to keep this module import-light (no detector
226         deps)."""
227         from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
228
229         vt, mg, mwd = preset.as_tuple()
230         return TrackNetRunner.trajectory_to_action_windows(
231             points,
232             fps=fps,
233             frame_width=frame_width,
234             velocity_thresh=vt,
235             merge_gap=mg,
236             min_window_duration=mwd,
237             max_window_duration=preset.max_window_duration,

```

```

233         window_hard_cap=preset.hard_cap,
234         window_inactivity_end=preset.inactivity_end,
235         inactivity_rally_thresh=preset.inactivity_rally_thresh,
236         inactivity_min_density=preset.inactivity_min_density,
237         inactivity_temporal_density=preset.inactivity_temporal_density,
238         window_blob_fallback=preset.blob_fallback,
239         window_blob_factor=preset.blob_factor,
240     )
241
242
243     def windows_to_intervals(windows: List[Dict[str, Any]]) -> List[Interval]:
244         """`[{start, end, ...}]` ? ``[(start, end)]` ? the interval shape the scorer/gate
speak."""
245         return [(w["start"], w["end"]) for w in windows]
246

```

16. assistant (2026-07-06T22:00:24.881Z)

This is the key. `build\_windows()` at `windowing.py:214-240` is the exact single call site that maps a `WindowingPreset` to `trajectory\_to\_action\_windows`. The `to\_dict()` at line 91-105 produces exactly the `preset` block shape seen in `nightly\_baseline.json`.

Now, one critical thing: the baseline JSON's `default`/`milestone` preset has `max\_window\_duration: 6.0` and `inactivity\_end: 1.5`, but the module-level `SERVED` preset is built from `IndexingConfig()` defaults. I need to confirm those config defaults. Let me check `models.py` (the `IndexingConfig`/`WasbConfig` fields) and how `rally\_seg\_eval.py` / `run\_regression.py` actually construct the `default`/`milestone` preset (whether they use module `SERVED` or build a custom preset with cap=6 + R4).

17. user (2026-07-06T22:00:27.502Z)

```

56:     velocity_threshold: float = 0.02
86:     velocity_threshold: float = 0.02
160:     # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
161:     # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
162:     velocity_threshold: float = 0.02
191:     dead_time_merge_gap: float = 2.0
196:     yolo_velocity_threshold: float = 0.15
200:     min_action_window_duration: float = 2.0
208:     max_window_duration: float = 6.0
214:     # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
216:     window_hard_cap: bool = False
220:     # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only cuts.
224:     window_inactivity_end: float = 1.5
225:     inactivity_rally_thresh: float = 0.20
226:     inactivity_min_density: float = 0.30
236:     inactivity_temporal_density: bool = True
239:     # than window_blob_factor × max_window_duration (no internal gap, no rally-end signal ?
e.g.
242:     # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
245:     window_blob_fallback: bool = False
246:     # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
250:     window_blob_factor: float = 4.0

```

18. user (2026-07-06T22:00:29.197Z)

```

10: windows under a :class:`~backend.eval.windowing.WindowingPreset` (so it tracks the SERVED
32: from backend.eval.windowing import PRESETS, SERVED, WindowingPreset
215:     video: Dict[str, Any], preset: WindowingPreset, min_crossings: int = 0
217:     """Parse a video's trajectory, window it under ``preset`` ? predicted rally intervals.
232:     wins = windowing.build_windows(points, video["fps"], video["frame_width"], preset)

```

```

275:     golden: List[Dict[str, Any]], preset: WindowingPreset, min_crossings: int = 0
288:         windows_for(v, preset, min_crossings),
314:     preset: WindowingPreset,
321:     """Score every golden video under ``preset`` (+ optional net-crossings gate); return
per-video
326:         predict_all(golden, preset, min_crossings),
333:         "preset": preset.to_dict(),
344:     base: WindowingPreset,
345:     cand: WindowingPreset,
402:     preset: WindowingPreset,
420:     predicted = predict_all(golden, preset, min_crossings)
439:         "preset": preset.to_dict(),
475:     p = result["preset"]
477:     if p.get("max_window_duration"):
479:         f"cap={p['max_window_duration']}" + ("(hard)" if p.get("hard_cap") else "")
481:     if p.get("inactivity_end"):
483:         f"inact_end={p['inactivity_end']}/rt={p['inactivity_rally_thresh']}"
567: def _resolve_preset(name: str) -> WindowingPreset:
570:     raise SystemExit(f"unknown preset '{name}'; choices: {sorted(PRESETS)}")
573: def _build_preset(args: argparse.Namespace, side: str) -> WindowingPreset:
574:     """Resolve the ``base`` (``--preset``) or ``cand`` (``--vs``) windowing from ``args`` by
575:     applying the ``dataclasses.replace`` overrides for cap / hard-cap / blob-fallback /
581:     preset = _resolve_preset(args.preset)
582:     cap = args.max_window_duration
583:     hard_cap = args.hard_cap
585:     inactivity_end = args.inactivity_end
588:     preset = _resolve_preset(args.vs)
589:     cap = (
590:         args.vs_max_window_duration
591:         if args.vs_max_window_duration is not None
592:         else args.max_window_duration
594:     hard_cap = args.vs_hard_cap or args.hard_cap
596:     inactivity_end = (
597:         args.vs_inactivity_end
598:         if args.vs_inactivity_end is not None
599:         else args.inactivity_end
604:     if cap is not None:
605:         preset = replace(preset, max_window_duration=cap)
606:     if hard_cap:
607:         preset = replace(preset, hard_cap=True)
609:         preset = replace(preset, blob_fallback=True)
611:         preset = replace(preset, inactivity_temporal_density=True)
613:         preset = replace(preset, blob_factor=args.blob_factor)
614:     if inactivity_end is not None:
615:         preset = replace(preset, inactivity_end=inactivity_end)
617:         preset = replace(preset, inactivity_rally_thresh=args.inactivity_rally_thresh)
619:         preset = replace(preset, inactivity_min_density=args.inactivity_min_density)
620:     return preset
630:     "--preset", default=SERVED.name, help="windowing preset (default: served)"
632:     ap.add_argument("--vs", default=None, help="second preset to diff against --preset")
638:     help="net-crossings rally-state gate on --preset (0=off; W2 Gate-A)",
650:     help="override bounded-windowing cap (s) on BOTH presets (W1; e.g. 12)",
656:     help="cap (s) on --vs ONLY (default: same as --max-window-duration); lets a "
657:     "BASELINE (no cap) vs CANDIDATE (capped) promotion check run in one shot",
660:     "--hard-cap",
662:     help="enforce the cap as a HARD cap on --preset (chop continuously-active "
663:     "windows at the cap when no inactivity gap exists; W1 hard-cap fallback)",
666:     "--vs-hard-cap",
668:     help="hard-cap fallback on --vs (default: same as --hard-cap)",
673:     help="R5 blob-fallback on --preset (after R4, force-chop any group still over the "
674:     "cap at its midpoint + flag/log it; the continuously-active blob last resort)",
685:     help="R5 magnitude gate (both presets): chop only groups > this × cap (default
4.0)",
691:     help="R4 rally-end inactivity trim (s) on --preset (0=off; trims post-rally drift
tail)",

```

```

703:         help="R4 velocity above which motion counts as 'rally' (both presets; default
0.08)",
709:         help="R4 min trailing fast-fraction to stay 'in rally' (both presets; default
0.34)",
715:         "count, on --preset (finding-B fix; no-op under dense tracking, trims sparse-track
tails)",
721:         "'--preset served --vs served --vs-inactivity-temporal-density' for the P2 promotion
A/B",
782:     base = _build_preset(args, "base")
785:     cand = _build_preset(args, "cand")
804:     f"\n## ? ({args.vs} mc={vs_mc} ? {args.preset} mc={args.min_crossings}), "
826:     # The long-rally payoff table always reports the BASE preset (the served/current
config),

```

19. assistant (2026-07-06T22:00:40.040Z)

This confirms the full picture. The ``default`/`milestone`` preset in ``nightly_baseline.json`` is exactly the module-level ``SERVED`` preset, which is built from ``IndexingConfig()`` defaults in ``models.py``. All the config defaults match the baseline JSON's ``preset`` block exactly:

```

- `velocity_threshold: 0.02` (line 86, wasb) ?
- `dead_time_merge_gap: 2.0` (line 191) ?
- `min_action_window_duration: 2.0` (line 200) ?
- `max_window_duration: 6.0` (line 208) ?
- `window_hard_cap: False` (line 216) ? `hard_cap: false` ?
- `window_inactivity_end: 1.5` (line 224) ? `inactivity_end: 1.5` ?
- `inactivity_rally_thresh: 0.20` (line 225) ?
- `inactivity_min_density: 0.30` (line 226) ?
- `inactivity_temporal_density: True` (line 236) ?
- `window_blob_fallback: False` (line 245) ? `blob_fallback: false` ?
- `window_blob_factor: 4.0` (line 250) ?

```

Let me confirm how the nightly regression actually invokes this (to be certain the ``default`` config == module ``SERVED``, not a config.json override), and check ``config.json`` for any windowing overrides. Let me also read the `models.py` context around `wasb velocity_threshold` to confirm which family the served preset uses.

20. user (2026-07-06T22:00:41.396Z)

```

1      """Nightly rally-quality regression tripwire.
2
3      Re-scores the golden corpus under HEAD via :mod:`backend.eval.rally_seg_eval` (CPU,
4      seconds ? the WASB GPU trajectories are CACHED, only the windowing/scorer changes) and
diffs
5      the headline metrics against a FROZEN baseline
(`eval_baselines/nightly_baseline.json`).
6      Writes a dated report and exits non-zero on any regression beyond tolerance OR any per-
video
7      over-segmentation-guard regression.
8
9      This is the owner's dual-eval-set discipline made automatic (`memory/eval-dual-set-
regression`)
10     + `docs/R2_EVAL_METRIC_DESIGN.md` §4.5): the growing GOLDEN-finalized set is re-run
every
11     night so a late change cannot silently lose an earlier win. It is default-
ADDITIVE ? it
12     does NOT change the promotion gate. It is a tripwire, not a re-training run: no GPU,
no Gemini,
13     no WASB re-extraction (those are cached and unchanged night to night).
14
15     Three tracked configs (all re-scored every night; a clean A/B/C on the same SERVED base
knobs):
16
17     * DEFAULT ? the SERVED production windowing, i.e. what users get today. Catches a
regression

```

```

18         to shipped behaviour. **NOTE (post-#204, 2026-06-18):** the R1 milestone is now
flipped ON, so
19     SERVED *is* cap=6 + R4 ? "default" now equals "milestone".
20     * **MILESTONE** ? the explicit R3+R4 config (cap=6 s + rally-END inactivity trim;
21         ``docs/QUALITY_ITERATIONS.md`` R3/R4). Now == DEFAULT (production shipped it); kept
explicit so
22         the nightly still names the milestone knobs and stays meaningful if production later
diverges.
23     * **BASELINE_OFF** ? the pre-R1-flip all-OFF windowing (every quality knob forced OFF).
The
24         permanent historical floor, so each report keeps showing the delta the milestone buys
(a launched
25         win must keep standing its ground, not be taken at launch-time value).
26
27     Ground truth + trajectories are LOCAL (the manifest points at owned, minors-free
trajectory CSVs
28         outside the repo + ``output/<name>_golden_features.json``); the corpus is NOT committed.
So the
29         baseline is tied to the owner's local golden corpus and is regenerated with ``--update-
baseline``
30         whenever the corpus grows or an intended improvement lands. (A cloud agent does not
suffice ? it
31         has the repo but not the cached trajectories; hence the local Windows Scheduled Task
wiring in
32         ``scripts/nightly_regression.ps1``.)
33
34     Usage
35     -----
36     Snapshot the baseline from current HEAD (after an intended improvement or a corpus
change):
37
38         python scripts/nightly_regression.py --update-baseline
39
40     Nightly check (exit 1 on regression; writes ``output/nightly_regression/<date>.md`` +
``.json``):
41
42         python scripts/nightly_regression.py
43
44     Exit codes
45     -----
46     * ``0`` ? all tracked configs clean (no regression), corpus matches the baseline.
47     * ``1`` ? at least one regression (an aggregate metric dropped beyond tolerance, or a
per-video
48         over-seg guard increased). Takes precedence over a stale baseline.
49     * ``2`` ? operational error (manifest/corpus missing, nothing scored).
50     * ``3`` ? no baseline to compare against (snapshot one with ``--update-baseline``).
51     * ``4`` ? baseline STALE (the corpus set or a config definition changed): a refresh is
needed,
52         but no regression was found on the comparable intersection.
53     """
54
55     from __future__ import annotations
56
57     import argparse
58     import datetime as _dt
59     import json
60     import os
61     import sys
62     from dataclasses import dataclass, field, replace
63     from typing import Any, Dict, List, Optional, Tuple
64
65     # ----- #
66     # Paths (resolved relative to the repo root, so the script is cwd-robust).
67     # ----- #
68     REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))

```

```

69     if REPO_ROOT not in sys.path:
70         sys.path.insert(
71             0, REPO_ROOT
72         ) # importable as `python scripts/nightly_regression.py` from anywhere
73     DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "nightly_baseline.json")
74     DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output", "nightly_regression")
75     DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "output", "human_lovo_manifest.json")
76     DEFAULT_FEATURES_DIR = os.path.join(REPO_ROOT, "output")
77     DEFAULT_IOU = 0.5
78
79     # ----- #
80     # What we gate on (a tripwire ? focused, not every metric).
81     # * HIGHER-better aggregates regress when they DROP beyond ``metric_tol``.
82     # * merge_rate (lower-better) regresses when it RISES beyond ``rate_tol``.
83     # * per-video split_count / merge_count must NOT increase beyond ``over_seg_tol``
84     # (the R2 §2.3.3 guard: "may not increase EITHER on ANY video").
85     # Boundary-error medians + seg_ratio are outlier-dominated diagnostics ? REPORTED, not
gated.
86     # ----- #
87     GATED_HIGHER: Tuple[str, ...] = ("tol_f1", "f1@0.5", "f1@0.25")
88     GATED_LOWER: Tuple[str, ...] = ("merge_rate",)
89     PER_VIDEO_GUARD: Tuple[str, ...] = ("split_count", "merge_count")
90
91     #: Aggregate metrics snapshotted into the baseline (the gated subset + report-only
context).
92     AGG_SNAPSHOT: Tuple[str, ...] = (
93         "tol_f1",
94         "tol_precision",
95         "tol_recall",
96         "f1@0.5",
97         "f1@0.25",
98         "precision",
99         "recall",
100        "merge_rate",
101        "split_rate",
102        "segment_count_ratio",
103        "dstart_abs_median",
104        "dend_abs_median",
105    )
106    #: Per-video metrics snapshotted into the baseline.
107    PV_SNAPSHOT: Tuple[str, ...] = (
108        "tol_f1",
109        "f1@0.5",
110        "f1@0.25",
111        "merge_rate",
112        "split_count",
113        "merge_count",
114        "num_pred",
115        "num_gt",
116    )
117
118    DEFAULT_METRIC_TOL = (
119        0.005 # F1-like drop that counts as a regression (absorbs float jitter)
120    )
121    DEFAULT_RATE_TOL = 0.010 # merge_rate rise that counts as a regression
122    DEFAULT_OVER_SEG_TOL = 0 # per-video split/merge increase allowed (strict)
123
124
125    # ----- #
126    # The two tracked configs.
127    # ----- #
128    @dataclass(frozen=True)
129    class NightlyConfig:
130        """A named windowing config the nightly re-scores (preset + net-crossings gate)."""
131

```

```

132     name: str
133     preset: Any # backend.eval.windowing.WindowingPreset
134     min_crossings: int = 0
135     note: str = ""
136
137
138     def tracked_configs() -> List[NightlyConfig]:
139         """DEFAULT (SERVED = shipped) + MILESTONE (explicit cap=6 + R4) + BASELINE_OFF (the
pre-flip floor).
140
141         All three are anchored on the SAME ``windowing.SERVED`` base knobs (velocity_thresh
/ merge_gap /
142         min_window) so they differ ONLY in the quality knobs ? they are a clean A/B/C:
143
144         * **DEFAULT** ? ``SERVED`` verbatim, so it can never silently drift from the served
operating
145         point. **Post-#204 (2026-06-18) the R1 milestone is flipped ON ? SERVED IS cap=6 +
R4, so
146         DEFAULT now equals MILESTONE.** Its job: catch an *accidental* change to a
production windowing
147         knob.
148         * **MILESTONE** ? SERVED + the explicit R3/R4 knobs (``docs/QUALITY_ITERATIONS.md``
R3/R4). Pins
149         the milestone *definition* independent of what production ships, so "is cap=6 + R4
still scoring
150         what we expect?" stays answerable even if a future change moves DEFAULT.
151         * **BASELINE_OFF** ? SERVED with every quality knob forced OFF (the pre-R1-flip
windowing). Pins
152         the *historical floor* so each nightly report keeps showing the delta the
milestone buys (the
153         dual-eval-set discipline: a launched win must keep standing its ground, not be
taken at
154         launch-time value). It is the reference DEFAULT used to be before the flip."""
155         from backend.eval.windowing import SERVED
156
157         default = NightlyConfig(
158             name="default",
159             preset=SERVED,
160             min_crossings=0,
161             note="SERVED production windowing ? now the R1 milestone (cap=6 + R4) after the
#204 flip.",
162         )
163         milestone = NightlyConfig(
164             name="milestone",
165             preset=replace(
166                 SERVED,
167                 name="milestone",
168                 max_window_duration=6.0,
169                 inactivity_end=1.5,
170                 inactivity_rally_thresh=0.20,
171                 inactivity_min_density=0.30,
172             ),
173             min_crossings=0,
174             note="R3 cap=6 s + R4 rally-END inactivity trim ? the shipped milestone (==
default post-#204).",
175         )
176         baseline_off = NightlyConfig(
177             name="baseline_off",
178             preset=replace(
179                 SERVED,
180                 name="baseline_off",
181                 max_window_duration=0.0,
182                 hard_cap=False,
183                 inactivity_end=0.0,
184                 inactivity_rally_thresh=0.08,

```

```

185         inactivity_min_density=0.34,
186         blob_fallback=False,
187         blob_factor=4.0,
188     ),
189     min_crossings=0,
190     note="Pre-R1-flip all-OFF windowing ? the historical floor; pins the value the
milestone buys.",
191 )
192     return [default, milestone, baseline_off]
193
194
195 # ----- #
196 # Tolerances bundle.
197 # ----- #
198 @dataclass(frozen=True)
199 class Tolerances:
200     metric_tol: float = DEFAULT_METRIC_TOL
201     rate_tol: float = DEFAULT_RATE_TOL
202     over_seg_tol: int = DEFAULT_OVER_SEG_TOL
203
204     def to_dict(self) -> Dict[str, Any]:
205         return {
206             "metric_tol": self.metric_tol,
207             "rate_tol": self.rate_tol,
208             "over_seg_tol": self.over_seg_tol,
209         }
210
211
212 # ----- #
213 # Pure summary + comparison core (no I/O ? unit-testable).
214 # ----- #
215 def summarize_run(eval_result: Dict[str, Any]) -> Dict[str, Any]:
216     """Compact, comparable summary of one :func:`rally_seg_eval.evaluate` result.
217
218     Keeps only the deterministic quantities we compare ? aggregate MEANS (not the
bootstrap CI,
219     which is seeded but irrelevant to the gate) and per-video RAW metric values."""
220     agg = eval_result.get("aggregate", {}) or {}
221     aggregate = {
222         k: (agg[k]["mean"] if isinstance(agg.get(k), dict) else None)
223         for k in AGG_SNAPSHOT
224     }
225     per_video = {
226         r["name"]: {k: r[k] for k in PV_SNAPSHOT if k in r}
227         for r in eval_result.get("rows", [])
228     }
229     return {
230         "n": int(agg.get("n", len(eval_result.get("rows", [])))),
231         "preset": eval_result.get("preset"),
232         "min_crossings": eval_result.get("min_crossings", 0),
233         "iou": eval_result.get("iou", DEFAULT_IOU),
234         "aggregate": aggregate,
235         "per_video": per_video,
236     }
237
238
239 @dataclass
240 class Finding:
241     """One comparison outcome line (regression / improvement / stale note)."""
242
243     config: str
244     scope: str # "aggregate" | "per_video" | "config"
245     metric: str
246     kind: str # "regression" | "improvement" | "stale"
247     baseline: Optional[float] = None

```

```

248     current: Optional[float] = None
249     video: Optional[str] = None
250
251     @property
252     def delta(self) -> Optional[float]:
253         if self.baseline is None or self.current is None:
254             return None
255         return self.current - self.baseline
256
257     def message(self) -> str:
258         loc = (
259             f"{self.config}/{self.video}/{self.metric}"
260             if self.video
261             else f"{self.config}/{self.metric}"
262         )
263         if self.kind == "stale":
264             return f"[STALE] {self.config}: {self.metric}"
265         d = self.delta
266         ds = f"{d:+.4f}" if d is not None else "?"
267         b = "?" if self.baseline is None else f"{self.baseline:.4f}"
268         c = "?" if self.current is None else f"{self.current:.4f}"
269         tag = "REGRESSION" if self.kind == "regression" else "improvement"
270         return f"[{tag}] {loc}: {b} ? {c} ({ds})"
271
272
273     @dataclass
274     class ConfigComparison:
275         config: str
276         status: str = "ok" # "ok" | "regression" | "stale"
277         findings: List[Finding] = field(default_factory=list)
278         added_videos: List[str] = field(default_factory=list)
279         removed_videos: List[str] = field(default_factory=list)
280         aggregate_gated: bool = (
281             True # False when preset/corpus changed ? aggregate not comparable
282         )
283
284         @property
285         def regressions(self) -> List[Finding]:
286             return [f for f in self.findings if f.kind == "regression"]
287
288         @property
289         def improvements(self) -> List[Finding]:
290             return [f for f in self.findings if f.kind == "improvement"]
291
292
293     def _preset_comparable(base_preset: Any, cur_preset: Any) -> bool:
294         """Are the two stored preset dicts the same windowing definition? (name is
295         cosmetic)."""
296         if not isinstance(base_preset, dict) or not isinstance(cur_preset, dict):
297             return base_preset == cur_preset
298         keys = set(base_preset) | set(cur_preset)
299         return all(base_preset.get(k) == cur_preset.get(k) for k in keys if k != "name")
300
301     def compare_config(
302         name: str, base_sum: Dict[str, Any], cur_sum: Dict[str, Any], tols: Tolerances
303     ) -> ConfigComparison:
304         """Compare one config's current summary to its baseline summary.
305
306         * If the preset definition or the net-crossings gate changed, the whole config is
307         STALE
308         (numbers aren't comparable) ? flag it, skip gating.
309         * If the corpus (video set) changed, the aggregate is not comparable (different
310         videos) ?
311         skip the aggregate gate + flag added/removed, but STILL run the per-video gate on

```

```

the
310     INTERSECTION (a code regression on a shared video must still trip the wire).
311     """
312     cmp = ConfigComparison(config=name)
313
314     # 1) Config-definition drift ? stale, not gateable.
315     if not _preset_comparable(
316         base_sum.get("preset"), cur_sum.get("preset")
317     ) or base_sum.get("min_crossings") != cur_sum.get("min_crossings"):
318         cmp.status = "stale"
319         cmp.aggregate_gated = False
320         cmp.findings.append(
321             Finding(config=name, scope="config", metric="preset", kind="stale")
322         )
323         return cmp
324
325     base_pv = base_sum.get("per_video", {})
326     cur_pv = cur_sum.get("per_video", {})
327     base_names, cur_names = set(base_pv), set(cur_pv)
328     cmp.added_videos = sorted(cur_names - base_names)
329     cmp.removed_videos = sorted(base_names - cur_names)
330     corpus_changed = bool(cmp.added_videos or cmp.removed_videos)
331
332     # 2) Aggregate gate (only when the corpus matches ? means over different sets aren't
comparable).
333     if corpus_changed:
334         cmp.aggregate_gated = False
335         cmp.status = "stale"
336         cmp.findings.append(
337             Finding(config=name, scope="config", metric="corpus", kind="stale")
338         )
339     else:
340         base_agg = base_sum.get("aggregate", {})
341         cur_agg = cur_sum.get("aggregate", {})
342         for m in GATED_HIGHER:
343             b, c = base_agg.get(m), cur_agg.get(m)
344             if b is None or c is None:
345                 continue
346             if c < b - tols.metric_tol:
347                 cmp.findings.append(Finding(name, "aggregate", m, "regression", b, c))
348             elif c > b + tols.metric_tol:
349                 cmp.findings.append(Finding(name, "aggregate", m, "improvement", b, c))
350         for m in GATED_LOWER:
351             b, c = base_agg.get(m), cur_agg.get(m)
352             if b is None or c is None:
353                 continue
354             if c > b + tols.rate_tol:
355                 cmp.findings.append(Finding(name, "aggregate", m, "regression", b, c))
356             elif c < b - tols.rate_tol:
357                 cmp.findings.append(Finding(name, "aggregate", m, "improvement", b, c))
358
359     # 3) Per-video over-seg guard on the intersection (always ? catches per-video code
regressions).
360     for vid in sorted(base_names & cur_names):
361         for m in PER_VIDEO_GUARD:
362             b, c = base_pv[vid].get(m), cur_pv[vid].get(m)
363             if b is None or c is None:
364                 continue
365             if c > b + tols.over_seg_tol:
366                 cmp.findings.append(
367                     Finding(
368                         name,
369                         "per_video",
370                         m,
371                         "regression",

```

```

372         float(b),
373         float(c),
374         video=vid,
375     )
376 )
377
378 if cmp.regressions:
379     cmp.status = "regression"
380 elif cmp.status != "stale":
381     cmp.status = "ok"
382 return cmp
383
384
385 @dataclass
386 class NightlyResult:
387     comparisons: List[ConfigComparison] = field(default_factory=list)
388     skipped: Dict[str, List[str]] = field(
389         default_factory=dict
390     ) # config -> manifest videos not scored
391
392 @property
393 def has_regression(self) -> bool:
394     return any(c.status == "regression" for c in self.comparisons)
395
396 @property
397 def has_stale(self) -> bool:
398     return any(c.status == "stale" for c in self.comparisons)
399
400 def overall_status(self) -> str:
401     if self.has_regression:
402         return "REGRESSION"
403     if self.has_stale:
404         return "STALE"
405     return "PASS"
406
407 def exit_code(self) -> int:
408     if self.has_regression:
409         return 1
410     if self.has_stale:
411         return 4
412     return 0
413
414
415 def compare_all(
416     baseline: Dict[str, Any],
417     current: Dict[str, Any],
418     tols: Tolerances,
419     skipped: Optional[Dict[str, List[str]]] = None,
420 ) -> NightlyResult:
421     """Compare every tracked config's current summary to the baseline summary."""
422     res = NightlyResult(skipped=skipped or {})
423     base_cfgs = baseline.get("configs", {})
424     cur_cfgs = current.get("configs", {})
425     for name in cur_cfgs:
426         if name not in base_cfgs:
427             cmp = ConfigComparison(config=name, status="stale", aggregate_gated=False)
428             cmp.findings.append(
429                 Finding(
430                     config=name,
431                     scope="config",
432                     metric="missing_in_baseline",
433                     kind="stale",
434                 )
435             )
436             res.comparisons.append(cmp)

```

```

437         continue
438     res.comparisons.append(
439         compare_config(name, base_cfgs[name], cur_cfgs[name], tols)
440     )
441     return res
442
443
444 # ----- #
445 # Report formatting (pure).
446 # ----- #
447 def _fmt(v: Optional[float]) -> str:
448     return "?" if v is None else f"{v:.4f}"
449
450
451 def format_report(
452     baseline: Dict[str, Any],
453     current: Dict[str, Any],
454     result: NightlyResult,
455     tols: Tolerances,
456     date: str,
457 ) -> str:
458     L: List[str] = []
459     status = result.overall_status()
460     badge = {
461         "PASS": "? PASS",
462         "REGRESSION": "? REGRESSION",
463         "STALE": "? STALE (baseline refresh needed)",
464     }[status]
465     L.append(f"# Nightly rally-quality regression ? {date}")
466     L.append("")
467     L.append(f"***Status: {badge}** · exit code {result.exit_code()}")
468     L.append("")
469     L.append(
470         f"- HEAD: `{current.get('git_commit', '?')}` · baseline: "
471         f"`{baseline.get('git_commit', '?')}` (snapshot {baseline.get('generated_at',
472         '?')})"
473     )
474     L.append(
475         f"- Corpus manifest: `{current.get('manifest', '?')}` · "
476         f"IoU={current.get('iou', DEFAULT_IOU)}"
477     )
478     L.append(
479         f"- Tolerances: metric ±{tols.metric_tol}, rate ±{tols.rate_tol}, "
480         f"per-video over-seg ±{tols.over_seg_tol}"
481     )
482     L.append("")
483
484 # Up-front regression / stale summary.
485 regs = [f for c in result.comparisons for f in c.regressions]
486 if regs:
487     L.append("## ? Regressions")
488     L.append("")
489     for f in regs:
490         L.append(f"- {f.message()}")
491     L.append("")
492 if result.has_stale:
493     L.append("## ? Stale / not comparable")
494     L.append("")
495     for c in result.comparisons:
496         if c.status == "stale":
497             if c.added_videos:
498                 L.append(f"- `{c.config}`: corpus added {c.added_videos}")
499             if c.removed_videos:
500                 L.append(f"- `{c.config}`: corpus removed {c.removed_videos}")
501             if not c.added_videos and not c.removed_videos:

```

```

500             L.append(
501                 f"- `{c.config}`: config definition changed vs baseline ? re-
snapshot with --update-baseline"
502             )
503             L.append(
504                 "- Action: `python scripts/nightly_regression.py --update-baseline` (after
confirming the change is intended)."
505             )
506             L.append("")
507
508             # Skipped (no silent caps).
509             skipped_any = {k: v for k, v in result.skipped.items() if v}
510             if skipped_any:
511                 L.append(
512                     "### ? Skipped manifest videos (missing trajectory or GT ? NOT silently
dropped)"
513                 )
514                 L.append("")
515                 for cfg, vids in skipped_any.items():
516                     L.append(f"- `{cfg}`: {vids}")
517                 L.append("")
518
519             # Per-config detail.
520             cur_cfgs = current.get("configs", {})
521             base_cfgs = baseline.get("configs", {})
522             for cmp in result.comparisons:
523                 cur = cur_cfgs.get(cmp.config, {})
524                 base = base_cfgs.get(cmp.config, {})
525                 L.append(f"### Config `{cmp.config}` ? {cur.get('note', '')}")
526                 L.append("")
527                 L.append(
528                     f"_status: {cmp.status}; n={cur.get('n', '?')} (baseline n={base.get('n',
'?'')})_"
529                 )
530                 L.append("")
531                 cagg = cur.get("aggregate", {})
532                 bagg = base.get("aggregate", {})
533                 L.append("| metric | baseline | current | ? | gated |")
534                 L.append("|---|---|---|---|---|")
535                 for m in AGG_SNAPSHOT:
536                     b, c = bagg.get(m), cagg.get(m)
537                     d = (c - b) if (b is not None and c is not None) else None
538                     gated = (
539                         "?"
540                         if (m in GATED_HIGHER or m in GATED_LOWER) and cmp.aggregate_gated
541                         else ""
542                     )
543                     L.append(f"| {m} | {_fmt(b)} | {_fmt(c)} | {_fmt(d)} | {gated} |")
544                 L.append("")
545             # Per-video over-seg guard table.
546             cpv = cur.get("per_video", {})
547             bpv = base.get("per_video", {})
548             names = sorted(set(cpv) | set(bpv))
549             if names:
550                 L.append("| video | tolF1 (b?c) | split (b?c) | merge (b?c) |")
551                 L.append("|---|---|---|---|")
552                 for v in names:
553                     cb, cc = bpv.get(v, {}), cpv.get(v, {})
554                     tf = f"{_fmt(cb.get('tol_f1'))}?{_fmt(cc.get('tol_f1'))}"
555                     sp = f"{cb.get('split_count', '?')}?{cc.get('split_count', '?')}"
556                     mg = f"{cb.get('merge_count', '?')}?{cc.get('merge_count', '?')}"
557                     flag = (
558                         " ?"
559                         if any(
560                             f.video == v and f.kind == "regression" for f in cmp.findings

```

```

561         )
562         else ""
563     )
564     L.append(f"| {v}{flag} | {tf} | {sp} | {mg} |")
565     L.append("")
566     imps = cmp.improvements
567     if imps:
568         L.append(
569             "_Improvements over baseline (consider `--update-baseline` to ratchet
the floor up):_"
570         )
571         for f in imps:
572             L.append(f"- {f.message()}")
573         L.append("")
574
575     L.append("---")
576     L.append(
577         "_Tripwire only ? re-scores CACHED trajectories (no GPU/Gemini/WASB). Does not
change "
578         "the promotion gate. See docs/R2_EVAL_METRIC_DESIGN.md + memory/eval-dual-set-
regression._"
579     )
580     return "\n".join(L)
581
582
583     # ----- #
584     # I/O shell.
585     # ----- #
586     def _git_commit() -> str:
587         import subprocess
588
589         try:
590             out = subprocess.run(
591                 ["git", "-C", REPO_ROOT, "rev-parse", "--short", "HEAD"],
592                 capture_output=True,
593                 text=True,
594                 timeout=10,
595             )
596             return out.stdout.strip() or "?"
597         except Exception:
598             return "?"
599
600
601     def _display_path(path: str) -> str:
602         """A clean, portable form for the committed baseline: ``output/<file>`` when the
path lives
603         in an ``output`` dir (the production layout), else repo-relative, else the path as
given.
604         Metadata only ? the comparison never reads it."""
605         norm = path.replace("\\", "/")
606         parent = os.path.basename(os.path.dirname(norm))
607         if parent == "output":
608             return "output/" + os.path.basename(norm)
609         try:
610             rel = os.path.relpath(path, REPO_ROOT)
611             if not rel.startswith(".."):
612                 return rel.replace("\\", "/")
613         except ValueError:
614             pass
615         return norm
616
617
618     def run_corpus(
619         configs: List[NightlyConfig],
620         manifest: str,

```

```

621     features_dir: str,
622     iou: float = DEFAULT_IΟΥ,
623 ) -> Tuple[Dict[str, Any], Dict[str, List[str]]]:
624     """Score every tracked config over the golden corpus. Returns (snapshot, skipped-
per-config).
625
626     ``snapshot`` is the serialisable structure stored as / compared to the baseline.
627     ``skipped``
628     lists manifest videos NOT scored (missing trajectory or empty GT) per config ?
629     surfaced in
630     the report so the corpus can't silently shrink."""
631     from backend.eval import rally_seg_eval
632
633     golden = rally_seg_eval.load_golden(manifest, features_dir)
634     manifest_names = [v["name"] for v in golden]
635     if not golden:
636         raise RuntimeError(f"no golden videos loaded from {manifest}")
637
638     snapshot: Dict[str, Any] = {
639         "manifest": _display_path(manifest),
640         "iou": iou,
641         "configs": {},
642     }
643     skipped: Dict[str, List[str]] = {}
644     scored_any = False
645     for cfg in configs:
646         result = rally_seg_eval.evaluate(
647             golden, cfg.preset, iou=iou, min_crossings=cfg.min_crossings
648         )
649         summary = summarize_run(result)
650         summary["note"] = cfg.note
651         snapshot["configs"][cfg.name] = summary
652         scored = set(summary["per_video"])
653         skipped[cfg.name] = [n for n in manifest_names if n not in scored]
654         scored_any = scored_any or bool(scored)
655     if not scored_any:
656         raise RuntimeError(
657             f"scored 0 videos for every config ? are the trajectory CSVs in {manifest}
present "
658             f"and do the {features_dir}/<name>_golden_features.json files have 'gts'?"
659         )
660     return snapshot, skipped
661
662 def load_baseline(path: str) -> Optional[Dict[str, Any]]:
663     if not os.path.exists(path):
664         return None
665     with open(path, encoding="utf-8") as fh:
666         return json.load(fh)
667
668 def save_baseline(path: str, snapshot: Dict[str, Any], tols: Tolerances) -> None:
669     out = dict(snapshot)
670     out["generated_at"] = _dt.date.today().isoformat()
671     out["git_commit"] = _git_commit()
672     out["tolerances"] = tols.to_dict()
673     out["_doc"] = (
674         "Frozen nightly rally-quality regression baseline. Regenerate with "
675         "`python scripts/nightly_regression.py --update-baseline` after an intended "
676         "improvement or a corpus change. Tied to the LOCAL golden corpus (not
committed)."
677     )
678     os.makedirs(os.path.dirname(path), exist_ok=True)
679     tmp = path + ".tmp"
680     with open(tmp, "w", encoding="utf-8") as fh:

```

```

681         json.dump(out, fh, indent=2, sort_keys=True)
682         fh.write("\n")
683     os.replace(tmp, path)
684
685
686 def build_parser() -> argparse.ArgumentParser:
687     p = argparse.ArgumentParser(
688         description="Nightly rally-quality regression tripwire (offline, CPU).")
689     )
690     p.add_argument("--manifest", default=DEFAULT_MANIFEST, help="golden manifest JSON")
691     p.add_argument(
692         "--features-dir",
693         default=DEFAULT_FEATURES_DIR,
694         help="dir holding <name>_golden_features.json",
695     )
696     p.add_argument(
697         "--baseline", default=DEFAULT_BASELINE, help="frozen baseline JSON path"
698     )
699     p.add_argument(
700         "--report-dir",
701         default=DEFAULT_REPORT_DIR,
702         help="where dated reports are written",
703     )
704     p.add_argument("--iou", type=float, default=DEFAULT_IOU)
705     p.add_argument(
706         "--metric-tol",
707         type=float,
708         default=DEFAULT_METRIC_TOL,
709         help="F1-like aggregate drop that counts as a regression",
710     )
711     p.add_argument(
712         "--rate-tol",
713         type=float,
714         default=DEFAULT_RATE_TOL,
715         help="merge_rate rise that counts as a regression",
716     )
717     p.add_argument(
718         "--over-seg-tol",
719         type=int,
720         default=DEFAULT_OVER_SEG_TOL,
721         help="per-video split/merge-count increase allowed before it's a regression",
722     )
723     p.add_argument(
724         "--update-baseline",
725         action="store_true",
726         help="snapshot the current numbers as the new baseline instead of checking",
727     )
728     p.add_argument(
729         "--date", default=None, help="override the report date (YYYY-MM-DD; for tests)"
730     )
731     p.add_argument(
732         "--no-report", action="store_true", help="skip writing the dated report file"
733     )
734     return p
735
736
737 def main(argv: Optional[List[str]] = None) -> int:
738     import logging
739
740     logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
741     log = logging.getLogger("nightly_regression")
742     args = build_parser().parse_args(argv)
743     tols = Tolerances(
744         metric_tol=args.metric_tol,
745         rate_tol=args.rate_tol,

```

```

746         over_seg_tol=args.over_seg_tol,
747     )
748     date = args.date or _dt.date.today().isoformat()
749
750     if not os.path.exists(args.manifest):
751         log.error(
752             "manifest not found: %s ? the golden corpus must be present locally.",
753             args.manifest,
754         )
755         return 2
756
757     try:
758         snapshot, skipped = run_corpus(
759             tracked_configs(), args.manifest, args.features_dir, args.iou
760         )
761     except RuntimeError as e:
762         log.error("%s", e)
763         return 2
764     snapshot["git_commit"] = _git_commit()
765     snapshot["generated_at"] = date
766
767     if args.update_baseline:
768         save_baseline(args.baseline, snapshot, tols)
769         log.info("baseline snapshot written to %s", args.baseline)
770         for cfg, s in snapshot["configs"].items():
771             agg = s["aggregate"]
772             log.info(
773                 " %s: n=%s tolF1=%%.3f f1@0.5=%%.3f merge_rate=%%.3f",
774                 cfg,
775                 s["n"],
776                 agg.get("tol_f1") or 0.0,
777                 agg.get("f1@0.5") or 0.0,
778                 agg.get("merge_rate") or 0.0,
779             )
780         return 0
781
782     baseline = load_baseline(args.baseline)
783     if baseline is None:
784         log.error(
785             "no baseline at %s ? snapshot one with --update-baseline (then commit it).",
786             args.baseline,
787         )
788         return 3
789
790     result = compare_all(baseline, snapshot, tols, skipped)
791     report = format_report(baseline, snapshot, result, tols, date)
792
793     if not args.no_report:
794         os.makedirs(args.report_dir, exist_ok=True)
795         md_path = os.path.join(args.report_dir, f"{date}.md")
796         json_path = os.path.join(args.report_dir, f"{date}.json")
797         with open(md_path, "w", encoding="utf-8") as fh:
798             fh.write(report + "\n")
799         with open(json_path, "w", encoding="utf-8") as fh:
800             json.dump(
801                 {
802                     "date": date,
803                     "status": result.overall_status(),
804                     "exit_code": result.exit_code(),
805                     "current": snapshot,
806                     "regressions": [
807                         f.message() for c in result.comparisons for f in c.regressions
808                     ],
809                 },
810                 fh,

```

```

811         indent=2,
812         sort_keys=True,
813     )
814     log.info("report written to %s", md_path)
815
816     print(report)
817     code = result.exit_code()
818     if code == 1:
819         log.error(
820             "NIGHTLY REGRESSION DETECTED (%d) ? see %s",
821             sum(len(c.regressions) for c in result.comparisons),
822             args.report_dir,
823         )
824     elif code == 4:
825         log.warning(
826             "baseline STALE ? corpus/config changed; refresh with --update-baseline"
827         )
828     else:
829         log.info("nightly regression check PASSED")
830     return code
831
832
833 if __name__ == "__main__":
834     raise SystemExit(main())
835

```

21. user (2026-07-06T22:00:42.060Z)

```

40     )
41     relevance: ValidationRelevanceConfig = Field(
42         default_factory=ValidationRelevanceConfig
43     )
44
45
46 class TrackNetConfig(BaseModel):
47     wsl_distro: str = "Ubuntu"
48     repo_dir: str = "~/models/TrackNetV4"
49     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
50     conda_env: str = "TrackNetV4"
51     weights_path: str = ""
52     queue_length: int = 5
53     stage_in_wsl: bool = True
54     wsl_stage_dir: str = "~/clips"
55     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
56     velocity_threshold: float = 0.02
57     # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
58     # decoded frame cache) are deleted automatically ? they are pure intermediates,
59     # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
60     # true only for debugging. On FAILURE they are always kept so a re-run can resume.
61     keep_frames: bool = False
62     # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of
the
63     # ~15x-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS,
not warns).
64     # 0 = no extra floor (the headroom guard still applies).
65     min_free_gb: float = 0.0
66
67
68 class WasbIndexConfig(BaseModel):
69     """Typed config for the live WASB shuttle substrate (indexing.wasb).
70
71     Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
72     these knobs actually take effect ? previously this whole block was silently dropped
73     because nested config sections defaulted to extra='ignore'.
74     """

```

```

75
76 wsl_distro: str = "Ubuntu"
77 repo_dir: str = "~/models/WASB-SBDT"
78 conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
79 conda_env: str = "wasb"
80 weights_path: str = (
81     "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
82 )
83 sport: str = "badminton"
84 wsl_stage_dir: str = "~/clips"
85 timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
86 velocity_threshold: float = 0.02
87 # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
88 # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
89 # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4
CPU fallback
90 # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-
CPU downgrade).
91 device: str = "cuda" # auto | cuda | mps | cpu
92 python_bin: str = (
93     "python" # the `wasb` conda env python (invoked by path, no `conda activate`)
94 )
95 # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
96 # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
97 # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
98 keep_frames: bool = False
99 # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of
the
100 # ~15x-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS,
not warns).
101 # 0 = no extra floor (the headroom guard still applies).
102 min_free_gb: float = 0.0
103 # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
104 # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
for a
105 # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
to the
106 # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
WSL disk,
107 # 1194/1195 frames byte-identical; native run-to-run byte-identical /
deterministic).
108 # Tri-state:
109 # None = per-runner default ? the WSL runner keeps DISK extraction
(unchanged Windows
110 # behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
111 # True/False = force on/off for BOTH runners (e.g. set False for a container cv2
can't seek).
112 stream_video: Optional[bool] = None
113 # GPU-feed tuning for the native runner (measured on the first real L4 serving run,
114 # 2026-07-03: the default batch 8 with single-process loading left the GPU <1%
utilized ?
115 # the pipeline was CPU-feed-bound). None = wasb_infer.py's parity defaults (batch 8,
no
116 # prefetch ? byte-identical to the historical path). The cloud-serving preset opts
into
117 # the measured fast values; local/WSL behaviour is unchanged unless set explicitly.
118 batch_size: Optional[int] = None
119 # Depth of the bounded producer-thread queue that overlaps CPU-side batch assembly
120 # (decode + PIL + ToTensor) with GPU inference in streaming mode. None/0 = off.
121 prefetch_batches: Optional[int] = None
122

```

```

123     @field_validator("batch_size")
124     @classmethod
125     def _batch_size_positive(cls, v: Optional[int]) -> Optional[int]:
126         # A DataLoader batch must be >=1; reject at config time so a typo can't reach
127         # `wasb_infer.py --batch-size` as an invalid value (the runner only passes the
flag
128         # when truthy, so 0 stays "off/parity-default").
129         if v is not None and v < 1:
130             raise ValueError(f"indexing.wasb.batch_size must be >= 1 when set (got
{v})")
131         return v
132
133     @field_validator("prefetch_batches")
134     @classmethod
135     def _prefetch_batches_non_negative(cls, v: Optional[int]) -> Optional[int]:
136         # 0 = off (the default). Reject negatives: the runner passes any truthy value as
137         # `--prefetch-batches`, and `_PrefetchIter` clamps depth with max(1,
int(depth)), so a
138         # negative would SILENTLY enable prefetch at depth 1 instead of failing loudly.
139         if v is not None and v < 0:

```

22. assistant (2026-07-06T22:00:49.663Z)

I now have complete confirmation. The `IndexingConfig` fields (lines 191?250 in models.py) are the source of truth for the `default`/`milestone` preset, and the nightly script (`tracked\_configs()`, lines 157?162) sets `default = SERVED` verbatim. Let me do one final verification: read the exact `IndexingConfig` field block (lines 185?250) to lock down the field-level defaults, and check `config.json` for any windowing overrides that would change what production actually serves.

23. user (2026-07-06T22:00:51.177Z)

```

185     # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
186     # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
187     # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
188     detector_impl: str = "auto"
189     motion_threshold: float = 0.08
190     min_segment_duration: float = 3.0
191     dead_time_merge_gap: float = 2.0
192     chunk_duration: float = 90.0
193     chunk_overlap: float = 10.0
194     detector_model: str = "fasterrcnn_resnet50_fpn"
195     detector_score_threshold: float = 0.5
196     yolo_velocity_threshold: float = 0.15
197     yolo_frame_skip: int = 3
198     yolo_tracker: str = "bytetrack.yaml"
199     yolo_prefetch_workers: int = 2
200     min_action_window_duration: float = 2.0
201     # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
202     # longer than this many seconds is recursively split at its largest internal
inactivity gap (?
203     # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
swallow
204     # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
205     # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
206     # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
207     # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
208     max_window_duration: float = 6.0
209     window_min_split_gap: float = (

```

```

210         0.5 # min internal gap (s) that qualifies as a split point
211     )
212     # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
213     # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
214     # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
215     # Only cuts (recall can't drop). OFF by default until measured/promoted.
216     window_hard_cap: bool = False
217     # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
218     # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
219     # [?end] gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
220     # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
221     # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
222     # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, [?end] 4.96?3.31s (n=13).
The W1 cap
223     # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
224     window_inactivity_end: float = 1.5
225     inactivity_rally_thresh: float = 0.20
226     inactivity_min_density: float = 0.30
227     # R4 density DENOMINATOR fix (code-review finding B). The R4 trim's fast-fraction
denominator is
228     # the COUNT of active samples in the trailing window, which sparse tracking inflates
(one fast
229     # sample after a gap ? 100% dense ? tail held open). When True, the denominator is
the window's
230     # TEMPORAL length instead. No-op under dense tracking (sample-count == temporal-
span), so it only
231     # corrects the sparse-track case. *** P2b PROMOTED DEFAULT-ON (2026-06-29): golden-
set A/B at the
232     # SERVED operating point (n=15, 595 rallies) ? tolF1 0.353?0.364 (?tolF1 +0.010,
sign 10W/4L, NO
233     # regression), [?end] 2.97?2.72s, seg_count_ratio ?0.103 (p=0.006), split_rate
?0.010 (p=0.016),
234     # R1 over-seg guard PASS; served_regresses(eps=0.0)=False ?
promote_if_served_safe(structural)=True.
235     # See docs/RALLY_DETECTION_FIXES_PLAN.md §7/§8. ***
236     inactivity_temporal_density: bool = True
237     # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
238     # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
239     # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
240     # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
241     # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
242     # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
243     # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
244     # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
245     window_blob_fallback: bool = False
246     # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
247     # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).

```

```

248 # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
249 # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
250 window_blob_factor: float = 4.0
251 log_yolo_candidates: bool = True
252 store_yolo_candidates: bool = True
253 ai_handoff_padding: float = 2.0
254 ai_max_workers: int = 5

```

24. user (2026-07-06T22:00:51.730Z)

```

1      {
2          "sport": "badminton",
3          "ai_provider": "gemini",
4          "ai_model": "gemini-flash-latest",
5          "validation": {
6              "min_resolution_width": 1280,
7              "min_resolution_height": 720,
8              "min_fps": 29.9,
9              "min_blur_threshold": 20.0,
10             "max_scene_cuts_per_minute": 0.5,
11             "relevance": {
12                 "court_green_lower": [
13                     35,
14                     40,
15                     40
16                 ],
17                 "court_green_upper": [
18                     85,
19                     255,
20                     255
21                 ],
22                 "court_pixels_min_ratio": 0.05
23             }
24         },
25         "indexing": {
26             "default_segmenter": "gemini",
27             "use_gpu": true,
28             "motion_threshold": 0.08,
29             "min_segment_duration": 3.0,
30             "dead_time_merge_gap": 2.0,
31             "chunk_duration": 90,
32             "chunk_overlap": 10,
33             "detector_model": "fasterrcnn_resnet50_fpn",
34             "detector_score_threshold": 0.5,
35             "yolo_velocity_threshold": 0.15,
36             "yolo_frame_skip": 3,
37             "yolo_tracker": "bytetrack.yaml",
38             "yolo_prefetch_workers": 2,
39             "min_action_window_duration": 2.0,
40             "log_yolo_candidates": true,
41             "store_yolo_candidates": true,
42             "ai_handoff_padding": 2.0,
43             "ai_max_workers": 1,
44             "tracknet": {
45                 "wsl_distro": "Ubuntu",
46                 "repo_dir": "~/models/TrackNetV4",
47                 "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
48                 "conda_env": "TrackNetV4",
49                 "weights_path": "",
50                 "queue_length": 5,
51                 "stage_in_wsl": true,
52                 "wsl_stage_dir": "~/clips",
53                 "timeout_sec": 1800,

```

```

54         "velocity_threshold": 0.02
55     },
56     "wasb": {
57         "wsl_distro": "Ubuntu",
58         "repo_dir": "~/models/WASB-SBDT",
59         "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
60         "conda_env": "wasb",
61         "weights_path": "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar",
62         "sport": "badminton",
63         "wsl_stage_dir": "~/clips",
64         "timeout_sec": 1800,
65         "velocity_threshold": 0.02
66     }
67 },
68 "output": {
69     "default_highlights_name": "highlights.mp4",
70     "db_name": "sports_indexer.db"
71 },
72 "stitching": {
73     "begin_padding": 0.5,
74     "end_padding": 0.5,
75     "use_cache": false,
76     "min_shot_count": 1,
77     "watermark": {
78         "enabled": true,
79         "text": "Generated by Khelsutra.guru",
80         "position": "bottom-right",
81         "opacity": 0.85,
82         "logo_path": "",
83         "font_path": ""
84     }
85 }
86 }

```

25. assistant (2026-07-06T22:00:58.073Z)

I have everything I need, fully verified. `config.json`'s `indexing` block does NOT override any of the windowing knobs (`max\_window\_duration`, `window\_inactivity\_end`, etc. are absent), so the `IndexingConfig` model defaults are exactly what `SERVED`/`default`/`milestone` use. Let me do one quick verification by actually constructing the preset to confirm the numbers match the baseline JSON byte-for-byte.

26. user (2026-07-06T22:01:01.023Z)

```

{
  "blob_factor": 4.0,
  "blob_fallback": false,
  "hard_cap": false,
  "inactivity_end": 1.5,
  "inactivity_min_density": 0.3,
  "inactivity_rally_thresh": 0.2,
  "inactivity_temporal_density": true,
  "max_window_duration": 6.0,
  "merge_gap": 2.0,
  "min_window_duration": 2.0,
  "name": "served",
  "velocity_thresh": 0.02
}

```

27. assistant (2026-07-06T22:01:38.794Z)

Confirmed byte-for-byte against `nightly\_baseline.json`'s `configs.default.preset` (only `name` differs: `served` vs the JSON's `served` for default / `milestone` for milestone ?

cosmetic). The runtime SERVED preset exactly reproduces the baseline's `default`/`milestone` windowing.

Here is the complete report.

Report: The `default`/`milestone` preset windowing configuration

Root cause of your degenerate scores

Your local scorer passes only `(velocity\_thresh=0.02, merge\_gap=2.0, min\_window\_duration=2.0)` and leaves everything else OFF. That is literally the `baseline_off` preset (baseline aggregate f1@0.5 = 0.237, and GX010128 = 0.060 ? matching your ~0.06). The `default`/`milestone` preset adds a `6-second segment cap + R4 rally-end inactivity trim`, which is what lifts GX010128 to 0.582 and aggregate to 0.472.

1. `trajectory\_to\_action\_windows` ? full signature

`backend/pipeline/detectors/tracknet\_runner.py:311-329` (static method on `TrackNetRunner`):

```
python
def trajectory_to_action_windows(
    points, fps, frame_width,
    chunk_start: float = 0.0,
    velocity_thresh: float = 0.02,
    merge_gap: float = 2.0,
    min_window_duration: float = 2.0,
    chunk_index: Optional[int] = None,
    max_window_duration: float = 0.0,          # segment CAP (0=OFF) ? split over-long windows at
largest inactivity gap
    min_split_gap: float = 0.5,                # min internal gap (s) that qualifies as a split
point
    window_hard_cap: bool = False,             # HARD cap: midpoint-chop gap-less over-long
windows
    window_inactivity_end: float = 0.0,        # R4 rally-END inactivity/dead-time trim (s, 0=OFF)
    inactivity_min_density: float = 0.34,      # R4: min trailing fast-fraction to stay "in rally"
    inactivity_rally_thresh: float = 0.08,     # R4: velocity above which motion counts as "rally"
    inactivity_temporal_density: bool = False, # R4 density denominator = temporal span vs
sample count
    window_blob_fallback: bool = False,        # R5 last-resort blob splitter (after R4)
    window_blob_factor: float = 4.0,          # R5 magnitude gate (x cap)
) -> List[Dict[str, Any]]:
```

Yes ? it supports `all` the knobs you asked about: segment `cap` (`max\_window\_duration`), hard cap (`window\_hard\_cap`), `inactivity/dead-time` end trim (`window\_inactivity\_end` + `inactivity\_rally\_thresh`/`inactivity\_min\_density`/`inactivity\_temporal\_density`), and an `over-seg blob guard` (`window\_blob\_fallback`/`window\_blob\_factor`). Note the parameter names differ from the JSON preset keys (JSON `hard_cap`?param `window_hard_cap`, JSON `inactivity_end`?param `window_inactivity_end`, JSON `blob_fallback`?`window_blob_fallback`, JSON `blob_factor`?`window_blob_factor`).

2. How `default`/`milestone` is defined

- The nightly baseline's tracked configs are built in `scripts/nightly_regression.py::tracked_configs()` (`lines 138-192`). `default` is `NightlyConfig(name="default", preset=SERVED)` `verbatim` (line 157-162). `milestone` is `SERVED` with the R3/R4 knobs re-stated explicitly (line 163-175) ? identical values, so `default == milestone`.
- `SERVED` is defined in `backend/eval/windowing.py:108-147` (`_served_preset_from_defaults`), which reads them straight out of `IndexingConfig()` defaults ? single-sourced so eval can't drift from production.
- The one call site that maps a preset ? the function is `backend/eval/windowing.py::build_windows()` (`lines 214-240`).
- `rally_seg_eval.evaluate` ? `predict_all` ? `windows_for` ? `build_windows(points, fps, frame_width, preset)` (`backend/eval/rally_seg_eval.py:232`).

- **``config.json` overrides nothing** ? its ``indexing`` block omits every windowing knob (``max_window_duration``, ``window_inactivity_end``, etc.), so the model defaults ARE the served operating point. Verified by constructing ``SERVED`` at runtime; its ``to_dict()`` matches ``nightly_baseline.json`` ``configs.default.preset`` byte-for-byte (only the cosmetic ``name`` differs).

3. The exact call to reproduce the ``default`` preset

```
```python
TrackNetRunner.trajectory_to_action_windows(
 points,
 fps=59.94,
 frame_width=1920,
 velocity_thresh=0.02,
 merge_gap=2.0,
 min_window_duration=2.0,
 max_window_duration=6.0, # cap = 6 s (R3) ? the dominant lever
 window_hard_cap=False,
 window_inactivity_end=1.5, # R4 rally-end trim
 inactivity_rally_thresh=0.20,
 inactivity_min_density=0.30,
 inactivity_temporal_density=True,
 window_blob_fallback=False,
 window_blob_factor=4.0,
)
```

(`min_split_gap` stays at its default `0.5`; it is not surfaced in the preset ? hardcoded via `window_min_split_gap` default in the model, also `0.5`.)
```

Easiest path: import and use the shipped preset directly rather than retyping ?

```
```python
from backend.eval.windowing import SERVED, build_windows
windows = build_windows(points, fps=59.94, frame_width=1920, preset=SERVED)
```
```

4. ``models.py`` config fields controlling windowing, and their defaults

```
`backend/config/models.py`, class `IndexingConfig`:
- `dead_time_merge_gap`: float = 2.0` (line 191) ? `merge_gap`
- `min_action_window_duration`: float = 2.0` (line 200) ? `min_window_duration`
- `max_window_duration`: float = 6.0` (line 208) ? **R1 milestone default-ON**
- `window_min_split_gap`: float = 0.5` (line 209) ? `min_split_gap`
- `window_hard_cap`: bool = False` (line 216)
- `window_inactivity_end`: float = 1.5` (line 224) ? **R1 milestone default-ON**
- `inactivity_rally_thresh`: float = 0.20` (line 225)
- `inactivity_min_density`: float = 0.30` (line 226)
- `inactivity_temporal_density`: bool = True` (line 236) ? **P2b promoted default-ON 2026-06-29**
- `window_blob_fallback`: bool = False` (line 245)
- `window_blob_factor`: float = 4.0` (line 250)
```

Velocity comes from the per-family sub-config: ``WasbIndexConfig.velocity_threshold``: float = 0.02` (``models.py:86``; the ``SERVED`` builder reads ``idx.wasb.velocity_threshold``). ``TrackNetConfig.velocity_threshold`` is also 0.02 (line 56).

****Which fields the "default preset" overrides vs the old all-OFF floor:**** relative to a naive ``(0.02, 2.0, 2.0)`` all-off windowing, the milestone flips ON ``max_window_duration=6.0``, ``window_inactivity_end=1.5``, ``inactivity_rally_thresh=0.20``, ``inactivity_min_density=0.30``, and ``inactivity_temporal_density=True``. ``window_hard_cap``, ``window_blob_fallback`` remain OFF; ``window_blob_factor=4.0`` is inert while ``blob_fallback`` is off.

Nothing left unpinned

All values are verified three ways: the frozen baseline JSON, the ``IndexingConfig`` model defaults, and a live construction of ``SERVED`` at HEAD (commit is clean, ``git pull`` was already up to date). The only caveat worth stating: the R4 trim's exact output also depends on ``min_split_gap=0.5`` (not in the preset dict but hardcoded to the model default), so include it if you re-implement the split logic yourself rather than calling

```
`build_windows`/`trajectory_to_action_windows`.
```